

TRƯỜNG ĐẠI HỌC ĐÀ LẠT
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO MÔN HỌC
THỰC HÀNH CÔNG NGHỆ PHẦN MỀM

Sinh viên thực hiện: Tạ Nhật Nguyên
Lớp: CTK47B
Mã số sinh viên: 2312704
Giảng viên hướng dẫn: Lê Gia Công

Đà Lạt, tháng 5 năm 2026

LỜI CẢM ƠN

Trong suốt quá trình học tập, nghiên cứu và hoàn thành tiểu luận môn Công nghệ phần mềm, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy Lê Gia Công.

Những bài giảng tâm huyết cùng hệ thống các bài học được Thầy biên soạn chi tiết trên blog không chỉ cung cấp cho em nền tảng lý thuyết vững chắc mà còn mang tính định hướng thực tiễn vô cùng lớn. Nhờ sự dẫn dắt và phương pháp giảng dạy trực quan của Thầy, em đã hiểu rõ hơn về bản chất của ngành kỹ thuật phần mềm. Thầy đã giúp em nhận ra rằng lập trình giỏi là chưa đủ, mà cốt lõi nằm ở quy trình, kiến trúc hệ thống và khả năng làm việc nhóm. Những kiến thức này đã giúp em trực tiếp tháo gỡ nhiều khó khăn khi triển khai các dự án thực tế của bản thân và nhóm trong thời gian qua.

Mặc dù đã cố gắng hết sức để tổng hợp kiến thức và liên hệ thực tiễn vào bài tiểu luận này, nhưng do trình độ chuyên môn và kinh nghiệm thực tế còn nhiều hạn chế, bài viết chắc chắn không thể tránh khỏi những thiếu sót. Em rất mong nhận được góp ý và chỉ bảo thêm từ Thầy để em có thể rút kinh nghiệm và hoàn thiện bản thân hơn trên con đường trở thành một kỹ sư phần mềm chuyên nghiệp.

Kính chúc Thầy luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp "trồng người" và truyền lửa đam mê cho các thế hệ sinh viên.

Em xin chân thành cảm ơn Thầy!

MỤC LỤC

MỞ ĐẦU.....	6
BÀI 1. GIỚI THIỆU VỀ CÔNG NGHỆ PHẦN MỀM.....	7
1.1. Yêu cầu bài tập	7
1.2. Báo cáo chi tiết	7
BÀI 2. CÁC MÔ HÌNH PHÁT TRIỂN PHẦN MỀM	9
2.1. Yêu cầu bài tập	9
2.2. Báo cáo chi tiết	9
2.2.1. Bài tập 2a - Sơ đồ các mô hình phát triển	9
2.2.2. Bài tập 2b – Các công việc trong mô hình Thác nước	12
BÀI 3. HỆ THỐNG QUẢN LÝ PHIÊN BẢN.....	14
3.1. Yêu cầu bài tập	14
3.2. Báo cáo chi tiết	14
3.2.1. Các phần mềm VSC phổ biến	14
3.2.2. Liên hệ thực tiễn – Áp dụng VCS vào các dự án phát triển phần mềm.....	15
BÀI 4. TỔNG QUAN VỀ GIT.....	16
4.1. Yêu cầu bài tập	16
4.2. Báo cáo chi tiết	16
4.2.1. Bài tập 2.1 – Cài đặt và cấu hình Git	16
4.2.2. Liên hệ thực tiễn - Áp dụng khởi tạo Git cho các dự án đang phát triển	17
BÀI 5. CẤU HÌNH ĐỊNH DANH NGƯỜI DÙNG.....	19
5.1. Yêu cầu bài tập	19
5.2. Báo cáo chi tiết	19
5.2.1. Thực hành cấu hình định danh các cấp	19
5.2.2. Bài tập 3.1 – Quản lý Git cho nhiều dự án.....	22
BÀI 6. CÁC KHU VỰC LÀM VIỆC CỦA GIT	24
6.1. Yêu cầu bài tập	24
6.2. Báo cáo chi tiết	24
6.2.1. Bài tập 4.1 – Di chuyển tệp qua 3 khu vực của Git	24
6.2.2. Liên hệ thực tiễn – Áp dụng vào dự án thực tế	26
BÀI 7. QUẢN LÝ DỰ ÁN PHẦN MỀM	28
7.1. Yêu cầu bài tập	28

7.2. Báo cáo chi tiết	28
7.2.1. Bài tập 3a - Vòng đời dự án ứng dụng PickleBook	28
7.2.2. Bài tập 3b - Bản Tôn chỉ dự án (Project Charter) và WBS	29
BÀI 8. CÁC KHU VỰC LÀM VIỆC CỦA GIT (TIẾP)	35
8.1. Yêu cầu bài tập	35
8.2. Báo cáo chi tiết	35
8.2.1. Bài tập 5.1 – Loại trừ tệp tin và lưu trữ commit	35
8.2.2. Liên hệ thực tiễn - Áp dụng .gitignore và commit trong dự án Unity	38
BÀI 9. LẤY VÀ PHÂN TÍCH YÊU CẦU	39
9.1. Yêu cầu bài tập	39
9.2. Báo cáo chi tiết	39
9.2.1. Bài tập 3a – Đặc tả yêu cầu người dùng	39
9.2.2. Bài tập 3b – Đặc tả yêu cầu phần mềm	43
BÀI 10. NGUYÊN TẮC LÀM VIỆC CỦA GIT	46
10.1. Yêu cầu bài tập	46
10.2. Báo cáo chi tiết	46
10.2.1. Bài tập 6a - Luân chuyển dữ liệu qua 3 khu vực	46
10.2.2. Bài tập 6b - Trạng thái tệp tin và lệnh git diff	47
10.2.3. Bài tập 6c - Xử lý sự cố và khôi phục dữ liệu	48
10.2.4. Bài tập 6d - Cơ chế lưu trữ của Git	50
BÀI 11. THIẾT KẾ KIẾN TRÚC PHẦN MỀM	51
11.1. Yêu cầu bài tập	51
11.2. Báo cáo chi tiết	51
11.2.1. Bài tập 5a - Kiến trúc 3-tầng cho dự án PickleBook	51
11.2.2. Bài tập 5b: Kiến trúc Vi dịch vụ cho dự án PickleBook	52
BÀI 12. LÀM QUEN VỚI DOCKER	54
12.1. Yêu cầu bài tập	54
12.2. Báo cáo chi tiết	54
BÀI 13. THIẾT KẾ CHI TIẾT	56
13.1. Yêu cầu bài tập	56
13.2. Báo cáo chi tiết	56
13.2.1. Thiết kế dữ liệu chi tiết	56

13.2.2. Thiết kế logic xử lý	56
13.2.3. Thiết kế giao diện lập trình API	57
13.2.4. Thiết kế phân cấp lớp OOP	58
BÀI 14. DOCKER IMAGE VÀ DOCKER CONTAINER.....	59
14.1. Yêu cầu bài tập	59
14.2. Báo cáo chi tiết.....	59
KẾT LUẬN	61

MỞ ĐẦU

Trong kỷ nguyên số hóa hiện nay, phần mềm đã trở thành "trái tim" vận hành mọi lĩnh vực của đời sống và kinh tế. Tuy nhiên, việc xây dựng một hệ thống đáp ứng được các tiêu chuẩn khắt khe về tính ổn định, bảo mật và khả năng mở rộng là một bài toán phức tạp. Chính vì vậy, Công nghệ phần mềm (Software Engineering) đóng vai trò then chốt khi cung cấp các phương pháp luận, quy trình quản lý và nguyên tắc kỹ thuật nhằm chuẩn hóa việc phát triển phần mềm thành một ngành công nghiệp có tính kỷ luật cao.

Là một sinh viên chuyên ngành Kỹ thuật phần mềm, việc tiếp thu và làm chủ những kiến thức này không chỉ là yêu cầu bắt buộc mà còn là hành trang thiết thực cho lộ trình nghề nghiệp tương lai. Bài tiểu luận này được thực hiện nhằm mục đích tổng kết, hệ thống hóa toàn bộ kiến thức đã được học qua 14 bài giảng của môn học Công nghệ phần mềm. Với định hướng "học đi đôi với hành", mục tiêu cốt lõi của bài làm không dừng lại ở việc tái hiện lý thuyết đơn thuần, mà là chứng minh khả năng thấu hiểu và vận dụng các quy trình đó vào thực tiễn.

Do đó, nội dung tiểu luận được cấu trúc một cách logic với ba trọng tâm chính cho mỗi bài học: hệ thống hóa các khái niệm nền tảng; trực tiếp đối chiếu, áp dụng lý thuyết vào quá trình phát triển các dự án thực tế (bao gồm ứng dụng di động, ứng dụng Game và nền tảng web); và cuối cùng là hoàn thiện các bài tập, báo cáo thực hành theo đúng yêu cầu.

Hy vọng thông qua cách tiếp cận này, tiểu luận sẽ phản ánh chính xác tinh thần cốt lõi của môn học, đồng thời thể hiện được thái độ làm việc nghiêm túc và sự nỗ lực trau dồi kỹ năng của bản thân trong suốt học kỳ vừa qua.

BÀI 1. GIỚI THIỆU VỀ CÔNG NGHỆ PHẦN MỀM

1.1. Yêu cầu bài tập

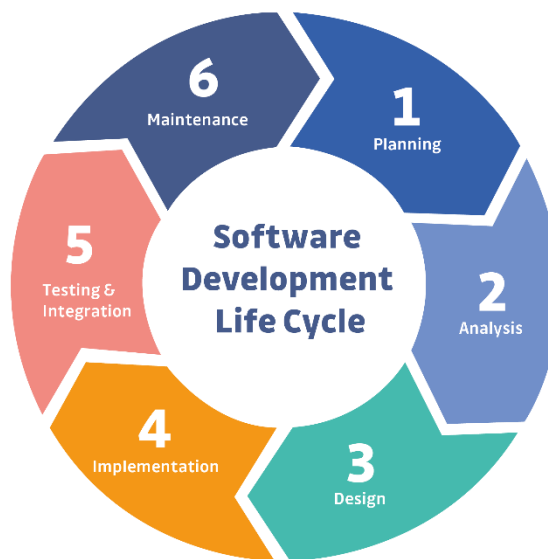
Bài tập 1: Bạn sẽ lựa chọn theo Công nghệ phần mềm hay Khoa học máy tính? Tại sao?

1.2. Báo cáo chi tiết

Để đưa ra quyết định lựa chọn giữa Công nghệ phần mềm (Software Engineering) và Khoa học máy tính (Computer Science), trước hết cần làm rõ đặc thù, vai trò và vòng đời phát triển của từng lĩnh vực khi áp dụng vào thực tế.

1. Cơ sở lựa chọn dựa trên quy trình sản xuất phần mềm

Công nghệ phần mềm không chỉ đơn thuần là viết mã lệnh (lập trình), mà là một ngành kỹ thuật bao trùm toàn bộ Vòng đời phát triển phần mềm (SDLC - Software Development Life Cycle).



Hình 1.1. Sơ đồ Vòng đời phát triển phần mềm (SDLC)

Như Hình 1.1 minh họa, để phát triển một phần mềm chất lượng, kỹ sư phải đi qua 6 bước (từ lấy yêu cầu, thiết kế, lập trình, kiểm thử, triển khai đến bảo trì). Kỹ năng lập trình xuất sắc là chưa đủ, mà giá trị cốt lõi nằm ở khả năng quản lý quy trình và đáp ứng 4 tiêu chuẩn chất lượng: tính dễ bảo trì, tính bảo mật, tính hiệu quả và tính dễ sử dụng. Khoa học máy tính sẽ giúp giải quyết các bài toán thuật toán phức tạp bên dưới, nhưng Công nghệ phần mềm mới là bộ quy chuẩn để đóng gói thuật toán đó thành sản phẩm hoàn chỉnh đến tay người dùng.

2. Phân tích dựa trên các dự án thực tiễn đang triển khai

Thông qua quá trình trực tiếp xây dựng các dự án hiện đang trong giai đoạn phát triển như ứng dụng di động PickleBook, tựa game 2D co-op Kermy (nhóm 5 thành viên) và ứng dụng Web TeoShop (bài tập cá nhân), em càng nhận thức rõ hơn tầm quan trọng của Công nghệ phần mềm.

Trong dự án Kermy, nhóm em nhận thấy rằng nếu chỉ dùng kiến thức Khoa học máy tính để tối ưu thuật toán xử lý va chạm vật lý cho game, dự án sẽ có nguy cơ thất bại nếu mã nguồn viết ra lộn xộn và 4 thành viên còn lại không thể đọc hiểu. Hiện tại, việc áp dụng Công nghệ phần mềm đang giúp nhóm thống nhất luồng xử lý giới hạn 2 người chơi rành mạch, chuẩn hóa quy tắc đặt tên biến và chia module từ sớm, nhằm đảm bảo "tính dễ bảo trì" và tối ưu hóa hiệu suất làm việc nhóm trong suốt vòng đời dự án.

Tương tự với ứng dụng Web TeoShop, do đang đảm nhiệm toàn bộ quy trình của một dự án cá nhân từ thiết kế cơ sở dữ liệu đến xử lý backend, Công nghệ phần mềm cung cấp cho em phương pháp để hệ thống hóa luồng dữ liệu ngay từ những bước đầu tiên. Điều này giúp em tích hợp các công nghệ một cách bài bản thay vì sa đà vào việc tự nghiên cứu và xây dựng lại các thuật toán lỗi đã có sẵn.

3. Kết luận về định hướng cá nhân

Từ những cơ sở lý thuyết và các vấn đề thực tiễn đúc kết được trong quá trình làm dự án nêu trên, em quyết định lựa chọn đi theo hướng Công nghệ phần mềm.

Lý do cốt lõi là em có đam mê lớn với việc tạo ra các sản phẩm ứng dụng thực tiễn. Em thích trải nghiệm xuyên suốt một quy trình sản xuất phần mềm chuyên nghiệp: từ lúc thu thập yêu cầu khách hàng, thiết kế kiến trúc hệ thống, cho đến khi kiểm thử và bảo trì. Khoa học máy tính trang bị cho em nền tảng tư duy logic, nhưng chính Công nghệ phần mềm mới cung cấp công cụ, bộ tiêu chuẩn và kỹ năng quản lý để giải quyết câu hỏi: "Làm sao để xây dựng một hệ thống phần mềm hoạt động ổn định và mang lại giá trị thực tế cho người dùng?". Đây chính là định hướng nghề nghiệp mà em quyết tâm theo đuổi và gắn bó lâu dài.

BÀI 2. CÁC MÔ HÌNH PHÁT TRIỂN PHẦN MỀM

2.1. Yêu cầu bài tập

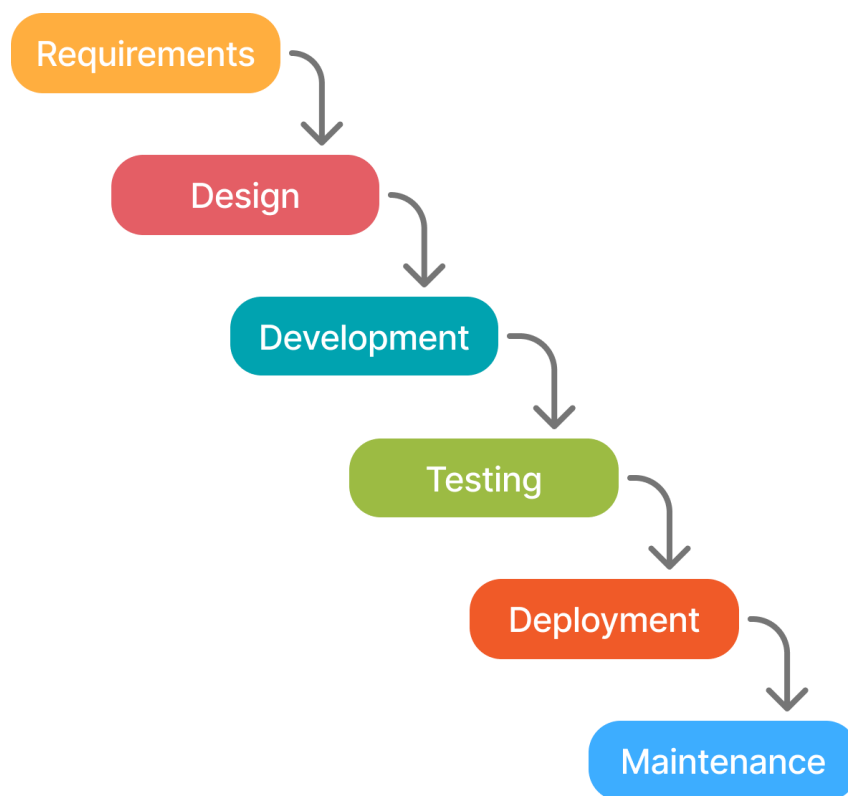
- Bài tập 2a: Vẽ lại sơ đồ các mô hình: Thác nước, chữ V, Tăng trưởng lặp và Agile/Scrum.
- Bài tập 2b: Mô tả công việc của mỗi công đoạn, công cụ sử dụng của mô hình Thác nước.

2.2. Báo cáo chi tiết

2.2.1. Bài tập 2a - Sơ đồ các mô hình phát triển

Dưới đây là sơ đồ 4 mô hình tiêu biểu kèm theo phân tích mức độ phù hợp đối với các dự án thực tiễn.

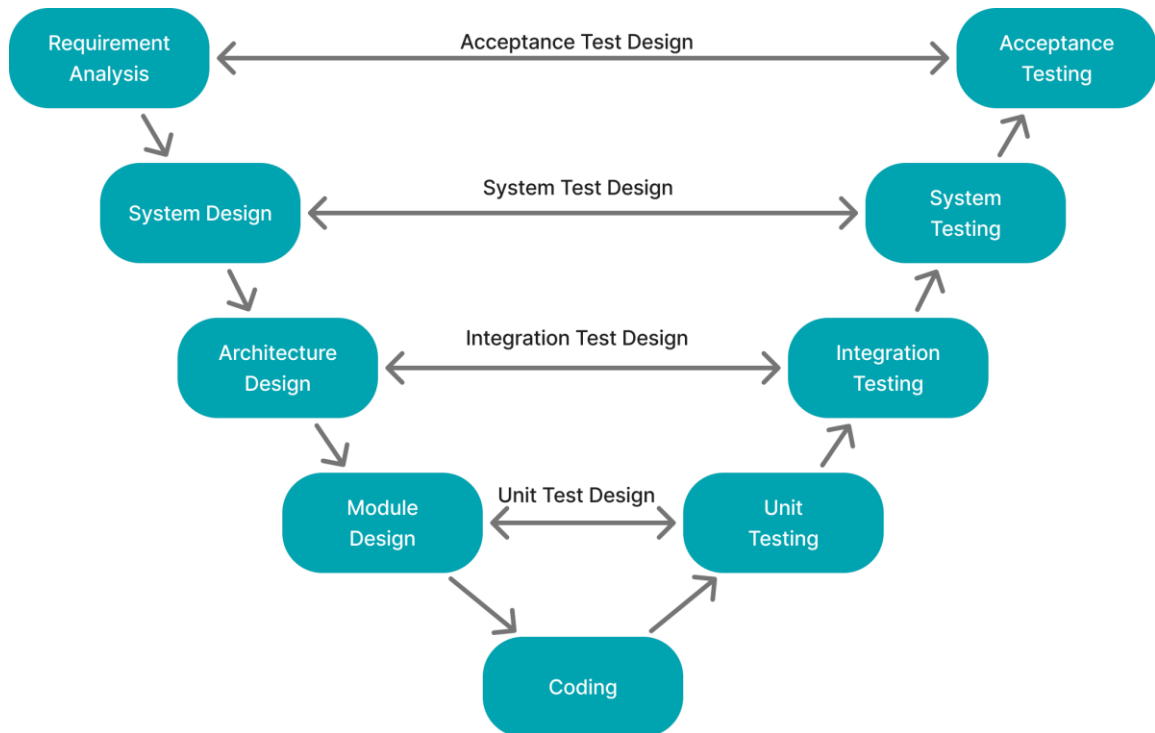
1. Mô hình Thác nước



Hình 2.1. Sơ đồ Mô hình Thác nước

Sơ đồ này cho thấy sự thiếu linh hoạt nếu áp dụng vào dự án thực tế. Ví dụ, trong quá trình nhóm 5 người làm game Kermy, nếu đang ở bước Lập trình mà phát hiện ra cơ chế vật lý (physics) thiết kế ban đầu không ổn, mô hình Thác nước không cho phép quay lại bước Thiết kế để sửa. Do đó mô hình này chỉ hợp với dự án có yêu cầu tĩnh.

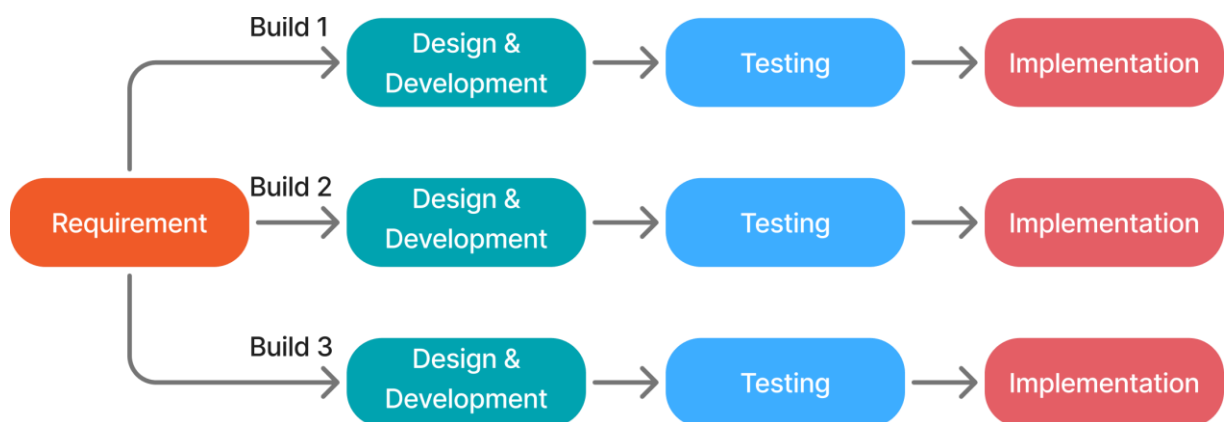
2. Mô hình chữ V



Hình 2.2. Sơ đồ Mô hình chữ V

Mô hình chữ V minh họa việc kiểm thử được thực hiện song song và cực kỳ chặt chẽ với từng bước thiết kế, giúp phần mềm rất an toàn và ít lỗi. Tuy nhiên, nó đòi hỏi phải viết quá nhiều tài liệu và cần nhiều nhân sự chuyên đi test lỗi. Do đó, mô hình này quá nặng nề, cứng nhắc và không hề phù hợp với những nhóm làm việc quy mô nhỏ 3-5 người như dự án PickleBook hay Kermy, vốn cần sự linh hoạt để thay đổi luồng xử lý liên tục.

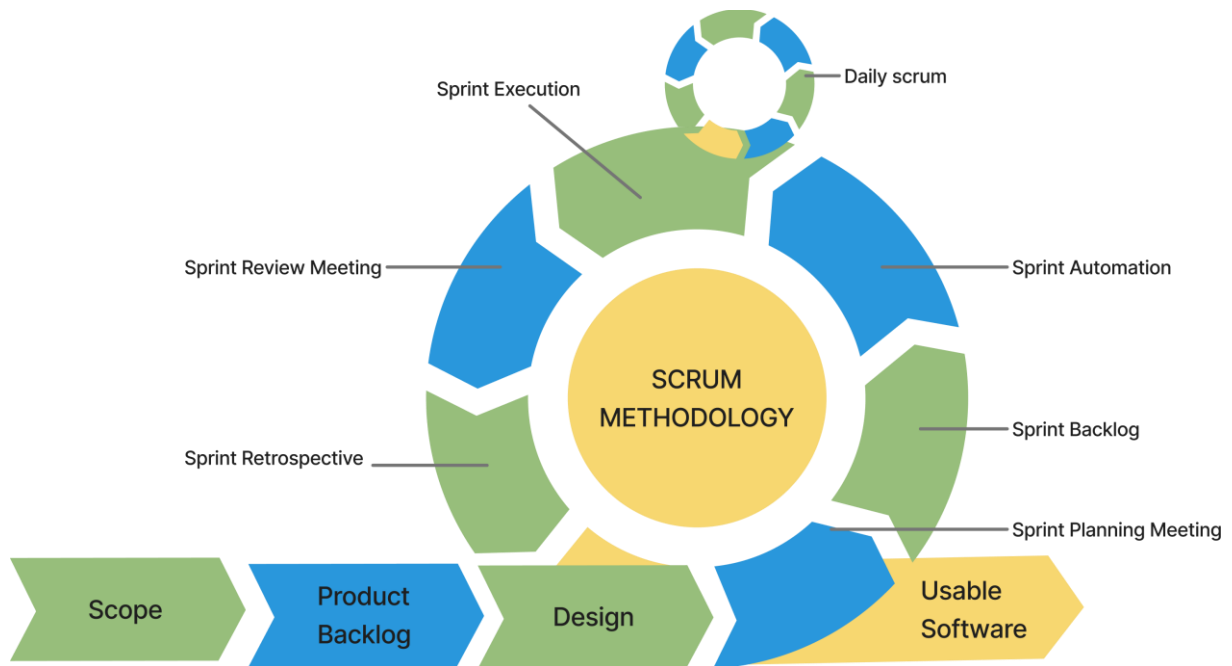
3. Mô hình Tăng trưởng lặp



Hình 2.3. Sơ đồ Mô hình Tăng trưởng lặp

Luồng hoạt động này thể hiện tính ứng dụng cao khi triển khai các dự án thực tế như ứng dụng web TeoShop. Thay vì phát triển toàn bộ hệ thống cùng lúc, quá trình xây dựng được chia nhỏ thành các vòng lặp chức năng: vòng lặp thứ nhất tập trung dựng khung nền tảng máy chủ với Node.js và Express; vòng lặp thứ hai tiến hành tích hợp cơ sở dữ liệu. Sơ đồ này đảm bảo việc luôn có sẵn một phiên bản phần mềm thực tế để đánh giá, kiểm thử và nghiệm thu vào cuối mỗi chu kỳ.

4. Mô hình Agile/Scrum



Hình 2.4. Sơ đồ Mô hình Agile/Scrum

Sơ đồ này phản ánh phương pháp luận quản lý tối ưu đối với các dự án đòi hỏi sự linh hoạt và làm việc nhóm tích cực như ứng dụng di động PickleBook. Các hạng mục công việc được phân rõ ràng; những module cốt lõi như hệ thống xác thực Firebase Auth hay nền tảng định vị Google Maps được đưa vào các Sprint riêng biệt để xử lý dứt điểm. Biểu đồ Scrum minh chứng cho tính linh hoạt tối đa, cho phép đội ngũ phát triển liên tục đánh giá tiến độ và tinh chỉnh lại kiến trúc hệ thống sau mỗi đợt làm việc ngắn hạn.

2.2.2. Bài tập 2b – Các công việc trong mô hình Thác nước

Mặc dù có nhiều hạn chế về tính linh hoạt, Mô hình Thác nước vẫn là nền tảng quy chuẩn để định nghĩa các đầu việc chi tiết trong một vòng đời phát triển phần mềm. Dưới đây là mô tả 6 công đoạn tuần tự cùng các công cụ hỗ trợ tương ứng:

[1] Giai đoạn Lấy và phân tích yêu cầu (Requirements Analysis)

- Mô tả công việc: Đây là giai đoạn nền móng nhằm xác định chính xác hệ thống phần mềm cần thực hiện những chức năng gì. Kỹ sư phần mềm sẽ làm việc trực tiếp với khách hàng để thu thập tính năng, đánh giá hiệu suất và phân loại thành yêu cầu chức năng hoặc phi chức năng. Kết quả đầu ra của giai đoạn này là tài liệu Đặc tả yêu cầu phần mềm (SRS - Software Requirements Specification).
- Công cụ hỗ trợ: Các phần mềm soạn thảo văn bản (Microsoft Word, Google Docs) hoặc các nền tảng không gian làm việc chung để quản lý tài liệu dự án (Confluence).

[2] Giai đoạn Thiết kế (Design)

- Mô tả công việc: Dựa trên tài liệu SRS, đội ngũ thiết kế sẽ chuyển hóa các yêu cầu thành bản vẽ kỹ thuật chi tiết. Quá trình này bao gồm việc thiết kế kiến trúc tổng thể (ví dụ: mô hình Client-Server), cấu trúc cơ sở dữ liệu, thuật toán lõi và xây dựng các sơ đồ UML (sơ đồ lớp, sơ đồ Use-case). Ngoài ra, thiết kế giao diện và trải nghiệm người dùng (UI/UX) cũng được hoàn thiện ở bước này.
- Công cụ hỗ trợ: Các công cụ thiết kế UML chuyên dụng (Enterprise Architect, Visual Paradigm, Lucidchart) và phần mềm thiết kế giao diện (Figma).

[3] Giai đoạn Lập trình (Coding/Implementation)

- Mô tả công việc: Lập trình viên sử dụng ngôn ngữ lập trình để hiện thực hóa các bản thiết kế thành mã nguồn thực tế. Trong giai đoạn này, yêu cầu bắt buộc là phải viết mã sạch (clean code), tuân thủ các nguyên lý kỹ thuật (SOLID, DRY, KISS), đặt tên biến theo chuẩn định trước và thiết lập hệ thống kiểm thử tự động (Unit test) song song với quá trình viết code.
- Công cụ hỗ trợ: Các môi trường phát triển tích hợp (IDE) như VS Code, IntelliJ, Android Studio. Đồng thời, sử dụng hệ thống quản lý phiên bản (Git) để kiểm soát thay đổi và đồng bộ mã nguồn khi làm việc nhóm.

[4] Giai đoạn Kiểm thử (Testing)

- Mô tả công việc: Đội ngũ Đảm bảo chất lượng (QA/QC) tiến hành tìm kiếm và khắc phục lỗi để phần mềm vận hành ổn định trước khi phát hành. Quá trình kiểm thử được thực hiện qua nhiều mức độ (Kiểm thử đơn vị - Unit, Kiểm thử tích hợp - Integration, Kiểm thử hệ thống - System, Kiểm thử chấp nhận - Acceptance) thông qua việc áp dụng các kỹ thuật hộp đen và hộp trắng.
- Công cụ hỗ trợ: Các công cụ kiểm thử tự động (Selenium, JMeter), hoặc sử dụng Postman để kiểm thử các luồng API (đặc biệt hiệu quả khi kiểm tra kết nối dữ liệu máy chủ).

[5] Giai đoạn Triển khai (Deployment)

- Mô tả công việc: Sản phẩm phần mềm sau khi vượt qua các bài kiểm thử khắt khe sẽ được đóng gói và cài đặt vào môi trường thực tế để bàn giao cho người dùng cuối. Giai đoạn này thường đi kèm với việc thiết lập các luồng Tích hợp và Triển khai liên tục (CI/CD) nhằm tự động hóa quy trình đưa mã nguồn lên hệ thống máy chủ.
- Công cụ hỗ trợ: Các nền tảng Container hóa và CI/CD (Docker, Jenkins, GitHub Actions) kết hợp cùng các dịch vụ điện toán đám mây (AWS, Azure, Firebase).

[6] Giai đoạn Vận hành và bảo trì (Maintenance)

- Mô tả công việc: Đây là giai đoạn kéo dài nhất trong vòng đời của một phần mềm. Đội ngũ kỹ thuật tiếp tục theo dõi hiệu năng, xử lý các lỗi phát sinh trong môi trường thực tế, cấu hình lại hệ thống để thích nghi với điều kiện mới và tiến hành tái cấu trúc mã nguồn (refactoring) nhằm tối ưu hóa khi lượng dữ liệu ngày càng phình to.
- Công cụ hỗ trợ: Các hệ thống giám sát hoạt động và cảnh báo (monitoring) như Grafana, Prometheus.

BÀI 3. HỆ THỐNG QUẢN LÝ PHIÊN BẢN

3.1. Yêu cầu bài tập

Bài 1.4: Tìm trang chủ của các phần mềm quản lý phiên bản (RCS, CVS, SVN, Git, Mercurial, Darcs).

3.2. Báo cáo chi tiết

Hệ thống quản lý phiên bản (Version Control Systems - VCS) là công cụ thiết yếu để lưu lại lịch sử thay đổi của mã nguồn dự án, hỗ trợ khôi phục phiên bản cũ, theo dõi người chỉnh sửa và giải quyết xung đột mã nguồn. Dựa trên kiến trúc hoạt động, VCS được chia làm 3 loại chính:

- Hệ thống quản lý phiên bản cục bộ (Local VCS): Lưu trữ các phiên bản thành các thư mục/tập tin khác nhau trên cùng một máy tính. Dễ bị sai sót, mất dữ liệu và không hỗ trợ làm việc nhóm.
- Hệ thống quản lý phiên bản tập trung (Centralized Version Control Systems - CVCS): Sử dụng một máy chủ trung tâm (Server) lưu trữ toàn bộ lịch sử dự án. Máy khách (Client) tải các tệp tin về để chỉnh sửa. Nhược điểm chí mạng là rủi ro mất dữ liệu nếu máy chủ trung tâm gặp sự cố và bắt buộc phải có kết nối mạng để lưu phiên bản mới.
- Hệ thống quản lý phiên bản phân tán (Distributed Version Control Systems - DVCS): Giải quyết triệt để hạn chế của CVCS. Mỗi máy khách không chỉ tải mã nguồn hiện tại mà sao chép (clone) toàn bộ kho chứa (repository) bao gồm cả lịch sử thay đổi. Cho phép thao tác offline và đảm bảo an toàn dữ liệu cao nhất.

3.2.1. Các phần mềm VSC phổ biến

Bảng 1. Danh sách các phần mềm quản lý phiên bản và địa chỉ trang chủ.

STT	Tên phần mềm	Phân loại	Trang chủ chính thức
1	RCS	Cục bộ	https://www.gnu.org/software/rcs/
2	CVS	Tập trung	http://nongnu.org/cvs/
3	Subversion (SVN)	Tập trung	https://subversion.apache.org/
4	Git	Phân tán	https://git-scm.com/

5	Mercurial	Phân tán	https://www.mercurial-scm.org/
6	Darcs	Phân tán	http://darcs.net/

3.2.2. Liên hệ thực tiễn – Áp dụng VCS vào các dự án phát triển phần mềm

Từ nền tảng lý thuyết trên, việc ứng dụng Hệ thống quản lý phiên bản phân tán (DVCS), cụ thể là công cụ Git, là tiêu chuẩn bắt buộc trong cả làm việc độc lập lẫn làm việc nhóm.

- Tránh rủi ro của quản lý phiên bản cục bộ (Áp dụng với dự án TeoShop):

Trước đây, đối với các bài tập nhỏ, việc lưu trữ theo kiểu VCS cục bộ (ví dụ: TeoShop_v1, TeoShop_v2). Cách làm này nhanh chóng dẫn đến sự hỗn loạn khi số lượng tệp tin mã nguồn tăng lên. Hiện tại, bằng việc khởi tạo repository cục bộ với Git cho dự án web TeoShop có thể theo dõi mọi thay đổi trong mã nguồn thông qua các commit, dễ dàng rẽ nhánh (branch) để thử nghiệm tính năng mới mà không làm ảnh hưởng đến mã nguồn chính đang hoạt động.

- Tối ưu hóa môi trường cộng tác nhiều người (Áp dụng với dự án Kermy và PickleBook):

Đối với dự án game 2D co-op Kermy (nhóm 5 thành viên) và ứng dụng PickleBook (nhóm 4 thành viên), việc sử dụng VCS cục bộ là bất thi, và DVCS tiềm ẩn rủi ro gián đoạn công việc nếu mạng không ổn định. Nhóm đã thống nhất áp dụng Git kết hợp nền tảng GitHub. Đúng với đặc trưng của hệ thống phân tán, mỗi thành viên trong nhóm đều clone toàn bộ repository về máy cá nhân (chứa đầy đủ lịch sử của cả nhóm). Điều này cho phép mỗi sinh viên tự viết mã và lưu phiên bản cục bộ ngay cả khi không có kết nối Internet (offline commit). Khi có mạng, các thành viên mới tiến hành push mã nguồn lên kho chứa từ xa (remote repository) và sử dụng các công cụ giải quyết xung đột (merge conflict) của Git để hợp nhất phần công việc của từng người thành một dự án hoàn chỉnh.

BÀI 4. TỔNG QUAN VỀ GIT

4.1. Yêu cầu bài tập

Bài tập 2.1: Thực hành các cài đặt trong bài học (bao gồm cài đặt phần mềm Git, kiểm tra phiên bản và nhúng Git vào thư mục dự án cục bộ).

4.2. Báo cáo chi tiết

Git là một Hệ thống quản lý phiên bản phân tán đóng vai trò xương sống trong quy trình phát triển phần mềm hiện đại. Git nổi bật với các tính năng cốt lõi như: theo dõi chi tiết lịch sử thay đổi mã nguồn, hỗ trợ phân nhánh (branch) để phát triển tính năng độc lập, quản lý hợp nhất (merge) và đảm bảo tính toàn vẹn của mã nguồn (thông qua các commit an toàn). Đặc biệt, cấu trúc phân tán cho phép mỗi lập trình viên sở hữu một bản sao kho chứa (repository) hoàn chỉnh, giúp việc phát triển phần mềm không bị phụ thuộc hoàn toàn vào kết nối mạng.

4.2.1. Bài tập 2.1 – Cài đặt và cấu hình Git

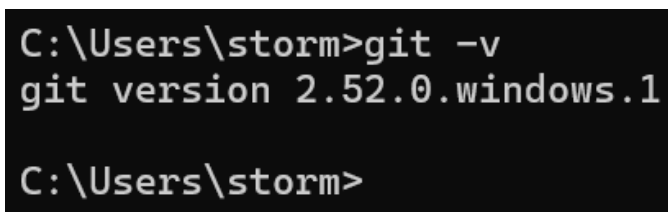
Dựa trên hướng dẫn của bài học, tiến hành tải, cài đặt và cấu hình Git trên môi trường máy tính cá nhân thông qua giao diện dòng lệnh (Command Line). Quá trình thực hành được ghi nhận qua các bước sau:

- Bước 1: Tải và cài đặt phần mềm

Truy cập trang chủ chính thức (<https://git-scm.com/>), tải bộ cài đặt phù hợp với hệ điều hành Windows và tiến hành cài đặt với các thiết lập mặc định.

- Bước 2: Kiểm tra trạng thái cài đặt

Mở cửa sổ lệnh (cmd), gõ dòng lệnh `git -v` để xác nhận phần mềm đã được cài đặt thành công và sẵn sàng hoạt động trong hệ thống biến môi trường (Environment Variables).



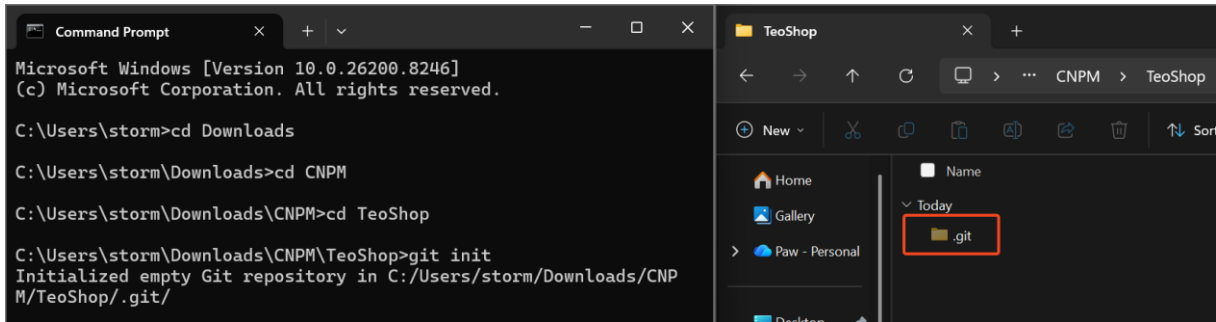
```
C:\Users\storm>git -v
git version 2.52.0.windows.1

C:\Users\storm>
```

Hình 4.1. Kiểm tra phiên bản Git trên máy tính

- Bước 3: Nhúng Git vào dự án (Khởi tạo Repository)

Sử dụng các lệnh điều hướng (địa:, cd) để di chuyển vào thư mục dự án trống. Sau đó, chạy lệnh git init. Lệnh này đã tự động sinh ra một thư mục ẩn mang tên .git – đây chính là kho chứa cục bộ (local repository) dùng để theo dõi mọi biến động của dự án từ thời điểm này trở đi.

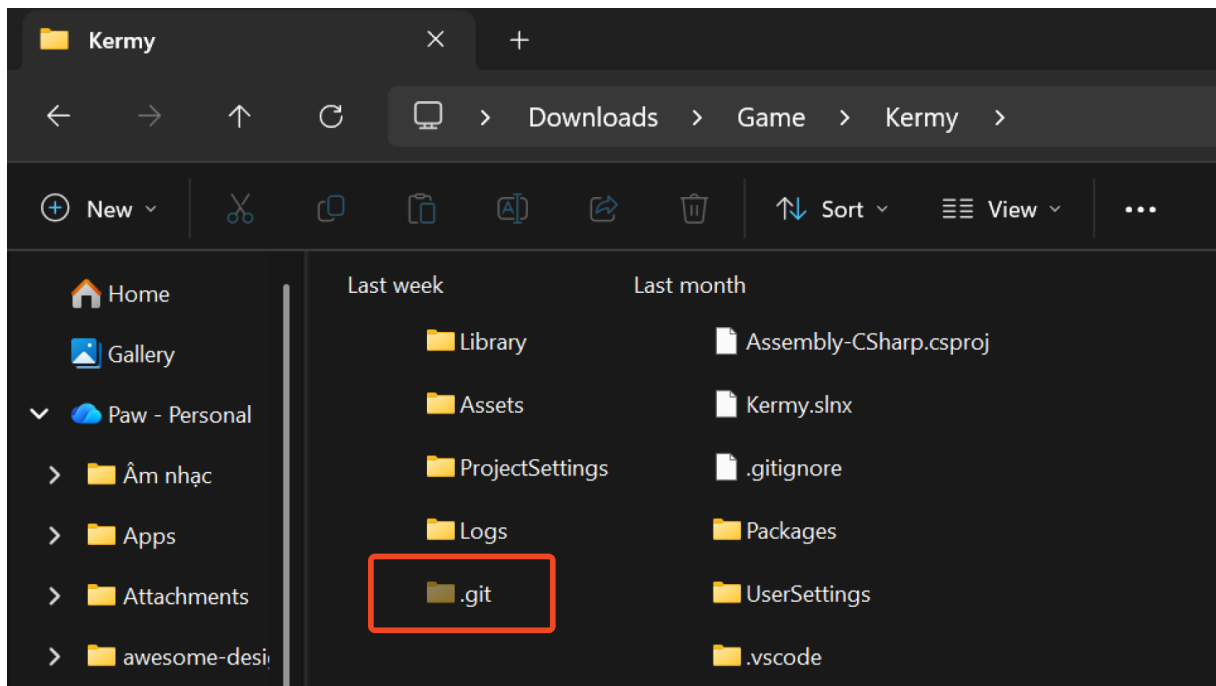


Hình 4.2. Khởi tạo thành công kho chứa Git trong thư mục dự án TeoShop

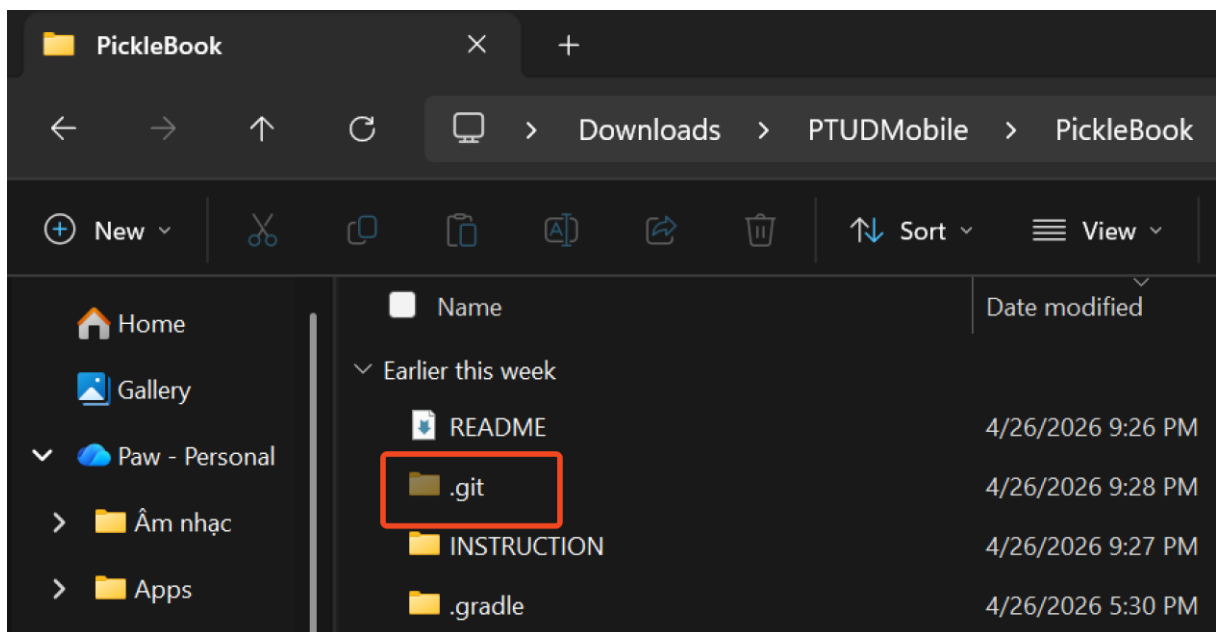
4.2.2. Liên hệ thực tiễn - Áp dụng khởi tạo Git cho các dự án đang phát triển

Việc gõ lệnh git init đối với một thư mục rỗng như ví dụ trong bài học rất đơn giản, nhưng khi áp dụng vào các dự án thực tế, quá trình khởi tạo kho chứa ngay từ những bước cấu trúc thư mục đầu tiên để đảm bảo mọi thay đổi đều được theo dõi:

Dự án game 2D Kermy (Unity) và PickleBook (Android): Trước khi nhóm tiến hành phân chia công việc và import các asset (tài nguyên) nặng vào Unity hay Android Studio, trưởng nhóm đã khởi tạo kho chứa cục bộ bằng git init ở thư mục gốc của dự án. Việc tạo kho chứa (Repository) ngay từ đầu giúp toàn bộ mã nguồn của nhóm có một nền tảng quản lý phiên bản thống nhất trước khi đẩy lên máy chủ.



Hình 4.3: Khởi tạo kho chứa Git cho dự án Kermy.



Hình 4.4. Khởi tạo kho chứa Git cho dự án PickleBook

BÀI 5. CẤU HÌNH ĐỊNH DANH NGƯỜI DÙNG

5.1. Yêu cầu bài tập

Thực hành: Thực thi các lệnh cấu hình định danh người dùng ở ba phạm vi (system, global, local) và kiểm tra thông tin cấu hình.

Bài tập 3.1 Bài tập tình huống “Quản lý cấu hình Git cho nhiều dự án”

Giải quyết tình huống “Quản lý cấu hình Git cho nhiều dự án”. Yêu cầu tạo hai không gian làm việc độc lập (TeoTechProject và VienVongProject) và cấu hình định danh (Tên, Email) riêng biệt cho từng dự án để tránh xung đột khi commit mã nguồn.

5.2. Báo cáo chi tiết

Để đảm bảo tính minh bạch, quy trách nhiệm và theo dõi đóng góp (đặc biệt trong làm việc nhóm), Git bắt buộc mọi sửa đổi (commit) phải được gắn với một định danh (Tên và Email). Hệ thống Git quản lý định danh qua ba mức độ ưu tiên:

- System (Phạm vi hệ thống): Áp dụng cho mọi tài khoản người dùng trên máy tính. Yêu cầu quyền Administrator.
- Global (Phạm vi người dùng): Áp dụng cho mọi kho chứa (repository) thuộc về tài khoản hệ điều hành hiện tại. Đây là cấu hình mặc định phổ biến nhất.
- Local (Phạm vi kho lưu trữ): Chỉ áp dụng riêng cho một thư mục dự án cụ thể. Cấu hình này có độ ưu tiên cao nhất, sẽ ghi đè lên thiết lập Global và System nếu có sự khác biệt.

5.2.1. Thực hành cấu hình định danh các cấp

- Thực hành cấu hình định danh ở phạm vi hệ thống (System):

Khi chạy lệnh `git config --system user.name "Van Teo"` trong cửa sổ Command Prompt bình thường, sẽ nhận được thông báo lỗi `Permission denied` đúng như bài học đề cập. Để khắc phục, đóng cmd, mở lại bằng quyền quản trị viên (Run as administrator) và thực thi lại lệnh. Kết quả lệnh chạy thành công và thông tin được lưu vào `C:\Program Files\Git\etc\gitconfig`.

```
C:\Users\storm>git config --system user.name "Van Teo"
error: could not lock config file C:/Program Files/Git/etc/gitconfig: Permission denied

git config --system user.name "Van Teo"
```

```
[init]
    defaultBranch = master
[user]
    name = Van Teo
```

Hình 5.1. Thiết lập cấu hình định danh ở phạm vi hệ thống

- Thực hành cấu hình định danh ở phạm vi người dùng (Global):
Tiếp tục ở cửa sổ cmd, việc cấu hình định danh cho toàn bộ tài khoản người dùng hiện tại bằng lệnh: `git config --global user.name "Le Van Teo"`

```
git config --global user.name "Le Van Teo"
```

```
[user]
    name = Le Van Teo
```

Hình 5.2. Thiết lập cấu hình định danh ở phạm vi người dùng

- Thực hành cấu hình định danh ở phạm vi kho lưu trữ (Local):
Di chuyển vào thư mục dự án TeoShop, thực hiện cấu hình định danh dành riêng cho dự án này và dùng lệnh xem danh sách để đối chiếu độ ưu tiên

```
TeoShop>git config user.name "Tran Le Van Teo"

TeoShop>git config --list
core.symlinks=false
core.ignorecase=true
user.name=Tran Le Van Teo
```

Hình 5.3. Thiết lập cấu hình định danh ở phạm vi kho lưu trữ

Kết quả hiển thị trong danh sách list cho thấy tên ở cuối cùng là "Tran Le Van Teo", chứng minh phạm vi kho lưu trữ (Local) đã đề thành công phạm vi người dùng (Global)

- Thực hành Thay đổi và Xóa định danh:

Trong quá trình làm việc, nếu phát hiện thông tin bị sai, có thể ghi đè (thay đổi) bằng cách gọi lại lệnh cấu hình với giá trị mới. Ví dụ thay đổi ở mức Global:

```
TeoShop>git config --global user.name "Ta Nhat Nguyen"
TeoShop>git config --global user.email "nhatnguyen@gmail.com"
TeoShop>git config --list
user.name=Ta Nhat Nguyen
user.email=nhatnguyen@gmail.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
```

Hình 5.4. Thay đổi định danh ở phạm vi người dùng

Trường hợp muốn gỡ bỏ hoàn toàn định danh đã thiết lập để không dính líu đến thông tin cũ, sử dụng tham số --unset:

```
TeoShop>git config --global --unset user.name
TeoShop>git config --global --unset user.email
TeoShop>git config --list
user.name=Van Teo
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Tran Le Van Teo
```

Hình 5.5. Xóa định danh ở phạm vi người dùng

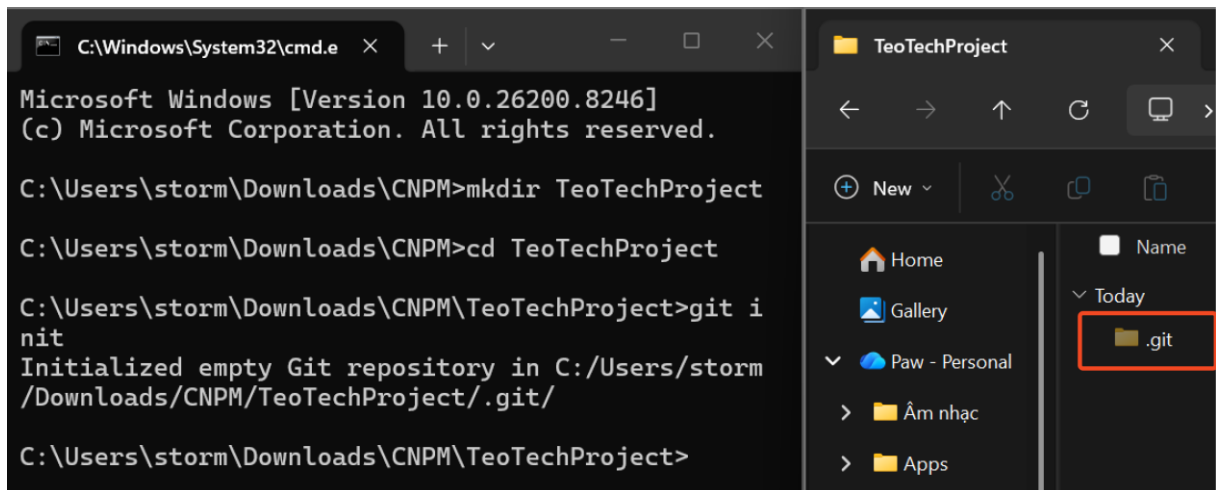
Kết quả hiển thị trong danh sách list cho thấy chỉ còn tên ở đầu là “Van Teo” (phạm vi hệ thống) và tên ở cuối cùng là "Tran Le Van Teo" (phạm vi kho lưu trữ), không còn phạm vi người dùng.

5.2.2. Bài tập 3.1 – Quản lý Git cho nhiều dự án

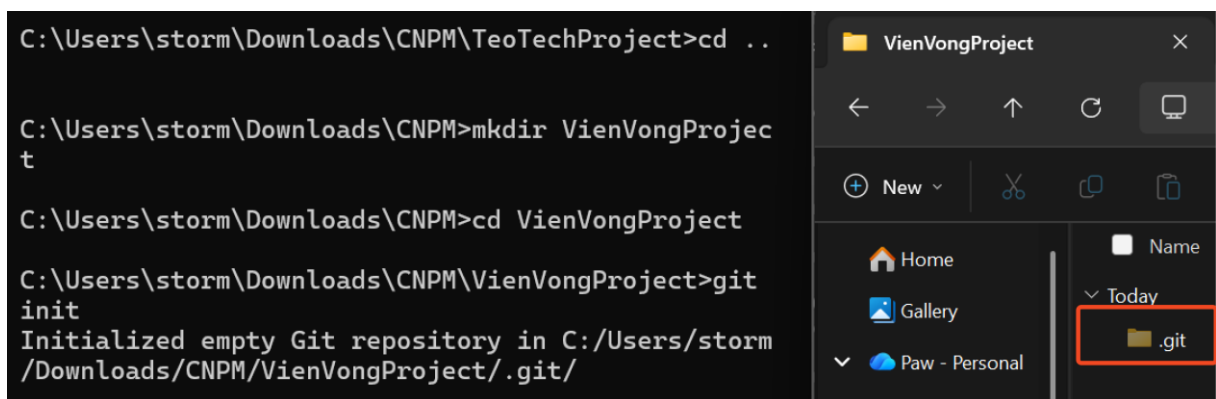
Để giải quyết bài toán một cách tối ưu, thay vì cấu hình thủ công cho từng dự án, vận dụng nguyên lý ưu tiên của Git: thiết lập thông tin cá nhân làm mặc định (Global) và chỉ ghi đè thông tin công ty cho dự án đặc thù (Local).

- Bước 1: Tạo thư mục và khởi tạo kho lưu trữ Git Đầu tiên

Tạo hai thư mục riêng biệt và dùng lệnh git init để khởi tạo kho lưu trữ rỗng cho từng dự án, di chuyển vào ổ đĩa làm việc, tạo và khởi tạo dự án công ty:



Hình 5.6. Khởi tạo dự án TeoTechProject và kho lưu trữ Git



Hình 5.7. Khởi tạo dự án VienVongProject và kho lưu trữ Git

- Bước 2: Thiết lập cấu hình và kiểm tra định danh cho các dự án

Để tối ưu hóa, thiết lập định danh cá nhân ở phạm vi người dùng (Global) làm mặc định và chỉ sử dụng cấu hình ở phạm vi kho lưu trữ (Local) để ghi đè cho dự án công ty. Tiến hành cấu hình và sử dụng lệnh git config --list để xác minh kết quả ngay lập tức cho từng dự án.

- Đối với dự án VienVongProject: Di chuyển vào thư mục VienVongProject, thiết lập định danh ở phạm vi người dùng (Global). Cấu hình này sẽ tự động áp dụng cho dự án này và mọi dự án cá nhân khác sau này.

```
VienVongProject>git config --global user.name "Nguyen Van Teo"
VienVongProject>git config --global user.email "nvteo@gmail.com"
VienVongProject>git config --list
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Nguyen Van Teo
user.email=nvteo@gmail.com
core.repositoryformatversion=0
core.filemode=false
```

Hình 5.8. Thiết lập cấu hình Global và kiểm tra định danh tại dự án VienVongProject

- Đối với dự án công ty (TeoTechProject): Để đảm bảo thông tin nộp cho công ty không bị lẫn email cá nhân, di chuyển sang thư mục `TeoTechProject` và thực hiện lệnh cấu hình ở phạm vi kho lưu trữ (Local) và không có tham số `--global`. Mức Local có độ ưu tiên cao nhất, giúp ghi đè cấu hình Global. Sau đó, kiểm tra lại danh sách cấu hình để xác nhận.

```
TeoTechProject>git config user.name "Nguyen Van Teo"
TeoTechProject>git config user.email "nvteo@teotech.com"
TeoTechProject>git config --list
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Nguyen Van Teo
user.email=nvteo@teotech.com
```

Hình 5.9. Thiết lập cấu hình Local (ghi đè Global) và kiểm tra định danh tại dự án TeoTechProject

BÀI 6. CÁC KHU VỰC LÀM VIỆC CỦA GIT

6.1. Yêu cầu bài tập

Bài tập 4.1: Thực hành lại các lệnh, tạo các thư mục, tập tin như trong bài học để hiểu rõ 3 khu vực làm việc của Git.

6.2. Báo cáo chi tiết

Khác với các hệ thống quản lý phiên bản khác, luồng xử lý cốt lõi của Git xoay quanh ba khu vực riêng biệt. Việc hiểu rõ 3 khu vực này là điều kiện tiên quyết để quản lý mã nguồn hiệu quả:

- Thư mục làm việc (Working Directory): Là thư mục dự án thực tế trên máy tính (chứa các tệp tin đang được chỉnh sửa). Đây là khu vực "sống", nơi thực hiện các thay đổi (thêm, sửa, xóa mã nguồn) trước khi đưa vào Git.
- Khu tạm (Staging Area): Là vùng đệm trung gian (được quản lý bởi tệp `.git/index`). Khu vực này cho phép chọn lọc chính xác những thay đổi nào sẽ được đóng gói cho lần lưu tiếp theo (bằng lệnh `git add`), thay vì phải lưu toàn bộ Thư mục làm việc.
- Kho chứa (Repository): Chính là thư mục ẩn `.git`. Đây là cơ sở dữ liệu lưu trữ vĩnh viễn toàn bộ lịch sử các phiên bản của dự án (sau khi dùng lệnh `git commit`).

6.2.1. Bài tập 4.1 – Di chuyển tệp qua 3 khu vực của Git

Tiến hành thực thi các lệnh `git status` và `git add` trên dự án TeoShop. Các bước thực hiện bao gồm:

- Bước 1: Kiểm tra trạng thái Thư mục làm việc ban đầu

Tại thư mục gốc của dự án TeoShop, tiến hành chạy lệnh `git status`. Do thư mục hiện tại hoàn toàn trống (ngoại trừ kho chứa `.git`), hệ thống trả về thông báo `nothing to commit`, xác nhận Thư mục làm việc chưa có sự thay đổi nào.


```
C:\Users\storm\Downloads\CNPM\TeoShop>git status
On branch master

No commits yet

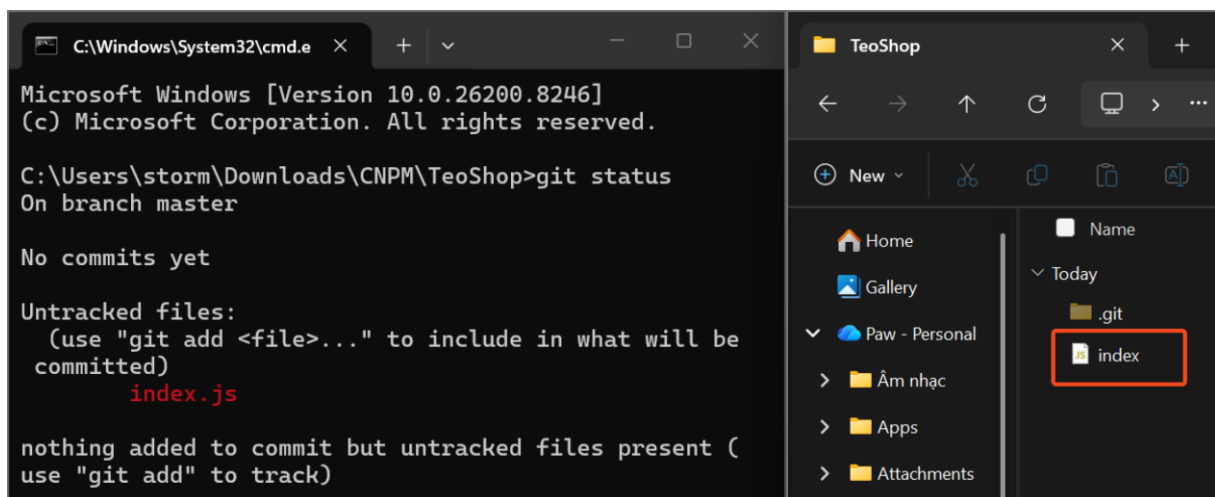
nothing to commit (create/copy files and use "git add" to track)

C:\Users\storm\Downloads\CNPM\TeoShop>|
```

Hình 6.1. Trạng thái ban đầu của dự án TeoShop khi chưa có tệp tin mã nguồn

- Bước 2: Khởi tạo tệp tin và kiểm tra trạng thái Untracked

Tiến hành tạo tệp `index.js` và bổ sung đoạn mã khởi tạo Web Server cơ bản bằng Express (Node.js). Ngay sau khi tệp tin được lưu, việc chạy lại lệnh `git status` cho thấy Git lập tức nhận diện `index.js` là một tệp tin mới (hiển thị dưới dạng Untracked files). Lúc này, tệp tin đang tồn tại ở Thư mục làm việc và chờ được xử lý.



Hình 6.2. Trạng thái `git status` nhận diện tệp `index.js` mới trong Thư mục làm việc.

- Bước 3: Chuyển tệp tin vào Khu tạm (Staging Area)

Để chuẩn bị lưu trữ phiên bản của tệp `index.js`, tiến hành thực thi lệnh `git add index.js`. Lệnh này đẩy tệp tin từ Thư mục làm việc sang Khu tạm. Việc kiểm tra lại bằng lệnh `git status` cho thấy trạng thái tệp tin đã chuyển thành Changes to be committed (hiển thị màu xanh lá), xác nhận tệp đã nằm trong Khu tạm và sẵn sàng để lưu vào Kho chứa.

```

C:\Users\storm\Downloads\CNPM\TeoShop>git add index.js

C:\Users\storm\Downloads\CNPM\TeoShop>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.js

```

Hình 6.3. Tập *index.js* được lệnh *git add* đưa vào Khu vực tạm thành công.

6.2.2. Liên hệ thực tiễn – Áp dụng vào dự án thực tế

Trong các dự án phức tạp như game 2D Kermey (Unity), một phiên làm việc thường làm thay đổi nhiều tệp tin cùng lúc. Kiểm tra bằng lệnh *git status* cho thấy hàng loạt kịch bản và tài nguyên đang ở trạng thái chờ tại Thư mục làm việc (màu đỏ - *Changes not staged for commit*).

```

C:\Users\storm\Downloads\Game\Kermey>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Assets/Art/Animation/Clips/Key.anim
        modified:   Assets/Script/Camera/CameraController.cs
        modified:   Assets/Script/Mechanics/BouncePad/BouncePad.cs
        modified:   Assets/_Scenes/Level01.unity

no changes added to commit (use "git add" and/or "git commit -a")

```

Hình 6.4. Thư mục làm việc với nhiều tệp tin bị thay đổi chưa được đưa vào Khu tạm.

Để tránh lưu trữ lẫn lộn cả những đoạn mã đang bị lỗi, Khu vực tạm (Staging Area) được sử dụng như một bộ lọc. Bằng cách thực thi lệnh *git add* chỉ định riêng cho tệp đã hoàn thiện chức năng (*CameraController.cs*), tệp này được chuyển thành công vào Khu tạm (màu xanh lá) để chuẩn bị đóng gói phiên bản. Các tệp chưa ổn định (*BouncePad.cs*) tiếp tục được cách ly an toàn tại Thư mục làm việc để tiến hành gỡ lỗi.

```
C:\Users\storm\Downloads\Game\Kermy>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   Assets/Script/Camera/CameraController.cs

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Assets/Art/Animation/Clips/Key.anim
        modified:   Assets/Script/Mechanics/BouncePad/BouncePad.cs
        modified:   Assets/_Scenes/Level01.unity
```

Hình 6.5. Sử dụng Khu tạm để chọn lọc tệp hoàn thiện và cách ly tệp chưa ổn định

BÀI 7. QUẢN LÝ DỰ ÁN PHẦN MỀM

7.1. Yêu cầu bài tập

- Bài tập 3a. Dự án của cá nhân
 - Mỗi cá nhân viết nội dung cho mỗi giai đoạn của dự án (theo mẫu Vòng đời của dự án)
 - Viết trên Microsoft Word
 - Mỗi bạn tạo một thư mục của cá nhân trên Google drive để chứa các kết quả làm việc của mình
 - Chia sẻ thư mục của cá nhân cho giáo viên (khi có yêu cầu)
- Bài tập 3b. Viết tên chỉ dự án
 - Tham khảo, viết bản tên chỉ dự án của cá nhân/nhóm [Mẫu]
 - Chép kết quả lên thư mục dự án của nhóm trên Google drive
- Bài tập 3c: Tự đọc thêm, cài đặt và trải nghiệm với Jira, Trello.

7.2. Báo cáo chi tiết

7.2.1. Bài tập 3a - Vòng đời dự án ứng dụng *PickleBook*

Dự án ứng dụng đặt lịch sân Pickleball (PickleBook) được chia thành 4 giai đoạn chính yếu trong vòng đời dự án như sau:

1. Giai đoạn Xác định nhiệm vụ (Initiation):

- Hoạt động: Phân tích nhu cầu của thị trường về phong trào chơi Pickleball đang lên nhưng thiếu nền tảng số hóa để kết nối người chơi và chủ sân. Xác định mục tiêu PCTS (Performance, Cost, Time, Scope).
- Kết quả: Thống nhất định hướng xây dựng ứng dụng trên nền tảng Android Native kết hợp Firebase. Hình thành nhóm 4 thành viên với các vị trí phụ trách 4 Module rõ ràng.

2. Giai đoạn Lập kế hoạch triển khai (Planning):

- Hoạt động: Phân rã cấu trúc công việc (WBS) thành 4 module lõi: Quản lý tài khoản (Auth), Bản đồ Google Maps, Xử lý Đặt lịch chống trùng giờ, và Bảng quản trị Admin.

- Kết quả: Chốt biểu đồ tiến độ thực hiện trong 4 tuần. Lựa chọn mô hình quản lý kho chứa mã nguồn (Git) và thống nhất các giao diện chức năng (UI/UX).

3. Giai đoạn Thực thi & Kiểm soát (Execution & Control):

- Hoạt động: Các thành viên bắt đầu viết code song song theo từng module được giao. Áp dụng quy tắc kiểm tra trùng lịch trên Firestore (Transactions). Trưởng nhóm giám sát mã nguồn trên Git để đảm bảo các module giao tiếp tron tru với nhau.
- Kết quả: Hoàn thành tính năng đặt sân theo đúng quy mô (Scope) và không bị vượt quá giới hạn thời gian (Time) đã định.

4. Giai đoạn Kết thúc dự án (Closure):

- Hoạt động: Kiểm thử toàn bộ luồng (Testing) từ lúc đăng ký đến lúc đặt sân và nhận thông báo. Sửa lỗi (fix bug) hiển thị trên màn hình nhỏ.
- Kết quả: Đóng gói mã nguồn. Xuất bản báo cáo kỹ thuật (Detailed Design Document) và bàn giao ứng dụng. Họp tổng kết rút kinh nghiệm nhóm.

7.2.2. Bài tập 3b - Bản Tôn chỉ dự án (Project Charter) và WBS

1. Tôn chỉ dự án (project charter)

1.0 Thông tin chung về dự án (project overview)	
Tên dự án (project name)	Phát triển Ứng dụng đặt lịch sân Pickleball trực tuyến (PickleBook) – Giai đoạn 1
Ngày bắt đầu (start date)	26/03/2026
Ngày kết thúc (end date)	26/05/2026
Tổng kinh phí của dự án (budget)	0 VNĐ (Dự án học thuật, tận dụng gói Firebase Spark Plan và nguồn lực nội bộ nhóm).
Quản lý dự án (PM – project manager)	Tạ Nhật Nguyên
Mô tả dự án	Bước đầu xây dựng hệ thống đặt lịch sân Pickleball qua ứng dụng di động: quy trình vận hành, quản lý sân bãi, tích hợp bản đồ và chống trùng giờ.
Đội dự án	04 thành viên (Sinh viên)

2.0 Bối cảnh
– Phong trào chơi thể thao Pickleball đang bùng nổ và phát triển rất mạnh mẽ tại Việt Nam. – Người chơi thường gặp khó khăn, mất nhiều thời gian trong việc tìm kiếm sân trống và đặt lịch (hiện tại chủ yếu phải gọi điện thoại hoặc nhắn tin Zalo thủ công). – Chủ sân quản lý lịch đặt sân bằng sổ sách hoặc file Excel dễ dẫn đến sai sót, trùng lịch khách hàng và khó tối ưu hóa tỷ lệ lấp đầy sân. – Nhóm sinh viên cần một dự án thực tế để áp dụng kiến thức về: thiết kế ứng dụng Android (Native), tích hợp Google Maps, quản lý cơ sở dữ liệu NoSQL (Firestore), xử lý xung đột dữ liệu (Booking Conflict) và quy trình phát triển phần mềm (SDLC/Agile).
3.0 Mục tiêu dự án
– Phát triển thành công hệ thống ứng dụng di động PickleBook chạy ổn định trên hệ điều hành Android. – Xây dựng hệ thống bản đồ trực quan giúp khách hàng dễ dàng tìm sân gần nhất. – Hoàn thiện thuật toán xử lý đặt lịch thời gian thực (Real-time), đảm bảo 100% không xảy ra tình trạng hai khách hàng đặt trùng một sân trong cùng một khung giờ. – Giúp chủ sân dễ dàng theo dõi, duyệt/hủy lịch qua công cụ quản lý trên app. – Giúp mỗi thành viên trong đội dự án hoàn thiện các kỹ năng: lập trình, cộng tác, giải quyết vấn đề, đúng giờ, tính kỉ luật, trách nhiệm.
4.0 Phạm vi dự án
– Xác thực & Quản lý tài khoản, Quản lý Sân & Bản đồ, Đặt lịch & Chống trùng giờ, Quản trị hệ thống & Thông báo. – Dữ liệu được lưu trữ và xử lý toàn bộ trên nền tảng Firebase. – Không bao gồm: Tích hợp cổng thanh toán ngân hàng trực tuyến (chỉ ghi nhận trạng thái Pending/Confirmed), không bao gồm phiên bản cho iOS, không kinh doanh dụng cụ thể thao.
5.0 Các kết quả chuyển giao
– Bộ mã nguồn (GitHub). – Tài liệu Đặc tả yêu cầu (SRS) và Thiết kế chi tiết (DDD). – File cài đặt ứng dụng Android (.apk). – Báo cáo tổng kết quá trình quản lý dự án.
6.0 Các mốc thời gian quan trọng
– Tuần 1: Hoàn thiện Charter, WBS và Module Xác thực người dùng (Auth). – Tuần 2: Tích hợp Bản đồ và Danh sách sân bãi. – Tuần 3: Xây dựng luồng Đặt sân và xử lý xung đột dữ liệu. – Tuần 4: Kiểm thử hệ thống, đóng gói sản phẩm và báo cáo.
7.0 Một số khó khăn
– Đội dự án làm việc bán thời gian (vừa học các môn khác vừa làm dự án).

– Chưa có nhiều kinh nghiệm tích hợp API bản đồ và xử lý giao dịch đồng thời (transaction) trên nền tảng NoSQL (Firebase).
8.0 Các rủi ro
– Biến động các thành viên trong nhóm (ốm đau, bận việc, thi cử trùng lịch). – Xung đột mã nguồn (Merge conflict) nghiêm trọng khi làm việc chung trên Git. – Rủi ro quá tải số lượng truy vấn miễn phí (quota) của Firebase trong quá trình test. – Rủi ro kỹ thuật khi xin cấp quyền truy cập API Google Maps (lỗi key, từ chối quyền).
9.0 Một số độ đo kết quả đạt được
– Tỷ lệ lỗi crash bằng 0 khi người dùng thực hiện luồng chức năng cơ bản. – Thời gian phản hồi của ứng dụng (load danh sách sân, hiển thị bản đồ) nhanh chóng. – Tỷ lệ xử lý trùng lịch thành công (100% chặn được 2 khách hàng đặt sân cùng lúc). – Đảm bảo tiến độ hoàn thành các module đúng hạn theo WBS.
10. Các yếu tố cần có để dự án thành công
– Tính kỉ luật, cam kết hoàn thành đúng deadline của mỗi thành viên trong nhóm. – đam mê công nghệ, chủ động học hỏi và nghiên cứu tài liệu về Android Native, Firebase. – Giao tiếp minh bạch, báo cáo tiến độ và khó khăn ngay lập tức trong nhóm chat Zalo để cùng giải quyết. – Phân công công việc rõ ràng, không để xảy ra tình trạng code đè lên nhau mà không báo trước.

2. Bảng phân rã công việc (WBS – work breakdown structure)

Tên công việc / Kết quả giao nộp	Chi tiết công việc	Phụ trách chính	Bắt đầu	Kết thúc
Khởi tạo và Lập kế hoạch dự án		Tạ Nhật Nguyên	26/03/2026	26/05/2026
	– Lập Tôn chỉ dự án (Project Charter) và bản WBS – Thiết kế kiến trúc hệ thống, sơ đồ CSDL (Firebase)			

	– Thiết lập cấu trúc dự án Android, tạo kho chứa GitHub			
Quản lý tài khoản (Auth)		Tạ Nhật Nguyên	26/03/2026	26/05/2026
	– Xây dựng giao diện Đăng nhập, Đăng ký, Quên mật khẩu – Tích hợp Firebase Authentication để xác thực người dùng – Cấu hình luồng phân quyền Role (Customer, Owner, Admin)			
Quản lý Sân và Bản đồ		Trần Quốc Khánh	26/03/2026	26/05/2026
	– Xây dựng giao diện Danh sách sân và Chi tiết sân bãi – Tích hợp Google Maps SDK, render vị trí sân bằng Marker – Hoàn thiện form Thêm/Sửa/Xóa thông tin sân (dành cho Chủ sân)			
Đặt lịch và Chống trùng giờ		Trần Gia Huy	26/03/2026	26/05/2026
	– Thiết kế giao diện lưới thời gian (Time Grid) để khách chọn giờ			

	– Viết thuật toán Transaction chống trùng lịch trên Firebase – Xây dựng luồng tính tiền và luồng duyệt/hủy đơn cho Chủ sân			
Quản trị và Thông báo		Nguyễn Nhật Minh	26/03/2026	26/05/2026
	– Cấu hình Dashboard thống kê cho tài khoản Admin – Tích hợp Firebase Cloud Messaging (FCM) gửi thông báo Push			
Kiểm thử và Đóng gói sản phẩm		Tạ Nhật Nguyên	26/03/2026	26/05/2026
	– Thực hiện kiểm thử toàn bộ hệ thống (End-to-End Testing) – Sửa lỗi (Fix bug) hiển thị trên các kích cỡ màn hình khác nhau – Xuất file cài đặt (.apk) và nộp báo cáo tiểu luận cuối kỳ			

3. Vai trò

STT	Họ tên	Phụ trách	Nhiệm vụ kiêm thêm
1	Tạ Nhật Nguyên	Quản lý chung	Quản lý tiến độ, vận hành GitHub, cấu hình Firebase gốc. Hỗ trợ test luồng dữ liệu.
2	Trần Quốc Khánh	Đội dự án	Cộng tác tích hợp luồng tìm sân từ Google Maps sang Đặt lịch. Hỗ trợ UI/UX.
3	Trần Gia Huy	Đội dự án	Tối ưu thuật toán Backend (Firestore), cộng tác xử lý luồng Booking chống trùng giờ.
4	Nguyễn Nhất Minh	Đội dự án	Kiểm thử hệ thống, quản trị Database, biên soạn tài liệu báo cáo kỹ thuật.

BÀI 8. CÁC KHU VỰC LÀM VIỆC CỦA GIT (TIẾP)

8.1. Yêu cầu bài tập

Bài tập 5.1: Thực hành các nội dung trong bài học (bao gồm thực hành các thao tác quản lý Khu tạm (git rm --cached), cấu hình loại trừ (.gitignore) và thực thi lệnh lưu trữ (git commit)).

8.2. Báo cáo chi tiết

Quá trình kiểm soát phiên bản trong Git đòi hỏi sự linh hoạt trong việc quản lý các tệp tin trước và sau khi lưu trữ. Các cơ chế cốt lõi bao gồm:

- Gỡ bỏ khỏi Khu tạm (Staging Area): Sử dụng lệnh git rm --cached <tên_tệp> để rút lại một tệp tin đã được thêm vào Khu tạm (bằng lệnh git add) trở về trạng thái Untracked tại Thư mục làm việc, mà không làm mất dữ liệu gốc của tệp.
- Cơ chế Git Ignore: Tập tin .gitignore chứa các quy tắc (pattern) chỉ định danh sách các tệp tin hoặc thư mục mà Git cần bỏ qua. Cơ chế này giúp loại trừ các tệp sinh ra tự động (log, file nhị phân, thư viện phụ thuộc) nhằm giữ cho Kho chứa (Repository) gọn nhẹ.
- Kho chứa (Repository) và Lệnh Commit: Kho chứa là cơ sở dữ liệu lưu trữ các thay đổi. Lệnh git commit đóng vai trò chuyển dữ liệu từ Khu tạm vào Kho chứa cục bộ (Local Repository), tạo ra một bản ghi (snapshot) đính kèm thông điệp (message) và định danh tác giả.

8.2.1. Bài tập 5.1 – Loại trừ tệp tin và lưu trữ commit

Tiến hành kiểm chứng các cơ chế quản lý mã nguồn thông qua môi trường dòng lệnh. Quá trình thực thi bao gồm các bước sau:

- Bước 1: Quản lý Khu tạm với lệnh git rm --cached

Tạo một tệp test.log và đưa vào Khu tạm bằng lệnh git add test.log. Sau đó, tiến hành gỡ tệp này khỏi Khu tạm bằng lệnh git rm --cached test.log. Việc kiểm tra lại bằng lệnh git status cho thấy tệp test.log đã quay trở lại trạng thái Untracked files.

```

C:\Users\storm\Downloads\CNPM\TeoShop>git rm --cached index.js
rm 'index.js'

C:\Users\storm\Downloads\CNPM\TeoShop>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.js

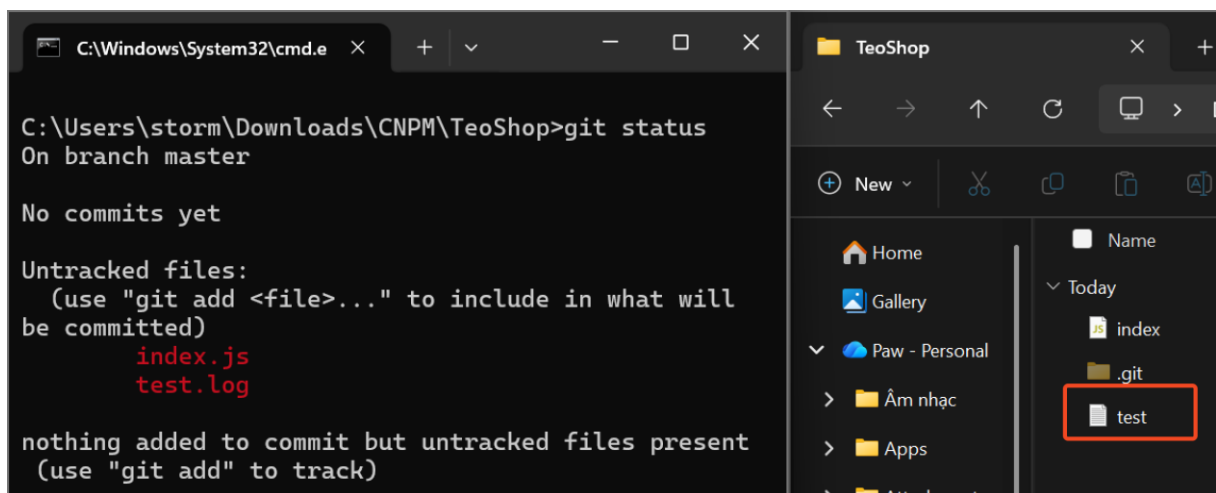
nothing added to commit but untracked files present (use "git add" to track)

```

Hình 8.1. Thực thi lệnh `git rm --cached` để gỡ bỏ tệp tin `index.js` khỏi Khu vực tạm.

- Bước 2: Thực hành cơ chế loại trừ với `.gitignore`

Tiến hành tạo một tệp tin rác mang tên `test.log` trong Thư mục làm việc. Kiểm tra bằng `git status`, hệ thống nhận diện `test.log` là một tệp tin mới (Untracked).



Hình 8.2. Git nhận diện tệp `test.log` khi chưa có cấu hình loại trừ.

Tiến hành kiểm chứng việc thêm và gỡ tệp tin `test.log`.

```

C:\Users\storm\Downloads\CNPM\TeoShop>git add test.log

C:\Users\storm\Downloads\CNPM\TeoShop>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   test.log

```

```
C:\Users\storm\Downloads\CNPM\TeoShop>git rm --cached test.log
rm 'test.log'

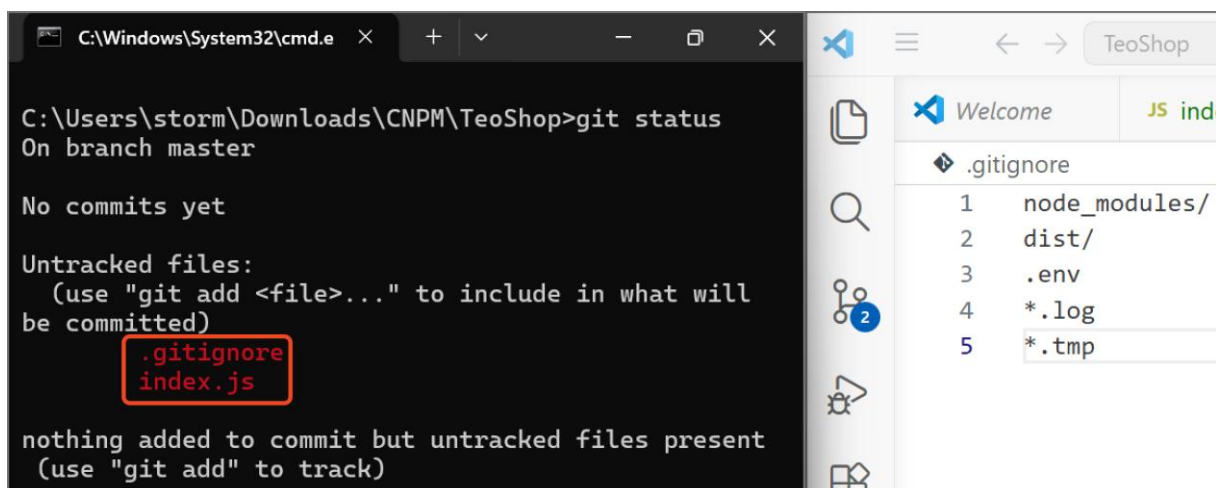
C:\Users\storm\Downloads\CNPM\TeoShop>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.js
    test.log
```

Hình 8.3. Đưa tệp tin test.log vào Khu tạm và gỡ bỏ trở lại Thư mục làm việc.

Để ngăn Git theo dõi các tệp nhật ký (log) này trong tương lai, tiến hành tạo tệp .gitignore tại thư mục gốc, sau đó chạy lại lệnh git status. Kết quả cho thấy Git chỉ còn thông báo về tệp .gitignore và index.js, hoàn toàn phớt lờ sự tồn tại của tệp test.log trong thư mục.



Hình 8.4. Cấu hình .gitignore để bỏ tệp tin test.log khỏi danh sách theo dõi của Git

- Bước 3: Thực thi lưu trữ bằng lệnh Commit

Để lưu trữ vĩnh viễn các trạng thái đã cấu hình, tiến hành đưa các tệp cần thiết vào Khu tạm và thực thi lệnh commit. Lệnh này đóng gói một "ảnh chụp" (snapshot) mới vào Kho chứa cục bộ, đi kèm với thông điệp ngắn gọn (Subject) mô tả công việc:

```

C:\Users\storm\Downloads\CNPM\TeoShop>git add index.js

C:\Users\storm\Downloads\CNPM\TeoShop>git add .gitignore

C:\Users\storm\Downloads\CNPM\TeoShop>git commit -m "Initialize Node.js project
with Express and Nodemon"
[master (root-commit) 2a3fe42] Initialize Node.js project with Express and Nodem
on
 2 files changed, 30 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 index.js

```

Hình 8.5. Thực thi lệnh git commit để lưu trữ vĩnh viễn các thay đổi vào Kho chứa

8.2.2. Liên hệ thực tiễn - Áp dụng .gitignore và commit trong dự án Unity

Việc nắm vững cơ chế loại trừ .gitignore và quy tắc lưu trữ Commit có giá trị đặc biệt quan trọng khi triển khai dự án game 2D (như Kermy) trên nền tảng Unity.

- Quản lý phiên bản với lệnh Commit: Lệnh commit tạo ra bản ghi (snapshot) lưu trữ thay đổi từ Khu tạm vào Kho chứa. Ví dụ, khi lưu trữ tệp CameraController.cs, lệnh git commit -m "Cập nhật logic bám đuôi nhân vật cho Camera" giúp các thành viên khác nhanh chóng nắm bắt tính năng mới mà không cần rà soát lại toàn bộ mã nguồn.

```

C:\Users\storm\Downloads\Game\Kermy>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   Assets/Script/Camera/CameraController.cs

C:\Users\storm\Downloads\Game\Kermy>git commit -m "Cập nhật
logic bám đuôi nhân vật cho Camera"
[main 17783e4] Cập nhật logic bám đuôi nhân vật cho Camera
 1 file changed, 42 insertions(+), 38 deletions(-)

```

Hình 8.6. Lưu trữ tệp CameraController.cs vào Kho chứa

BÀI 9. LẤY VÀ PHÂN TÍCH YÊU CẦU

9.1. Yêu cầu bài tập

Bài tập 3a. Dựa vào Tài liệu đặc tả yêu cầu người dùng (User Requirement Specification - URS). Viết URS cho dự án của bạn (nhóm).

Bài tập 3b. Từ URS trên chuyển thành SRS.

9.2. Báo cáo chi tiết

9.2.1. Bài tập 3a – Đặc tả yêu cầu người dùng

URS - TÀI LIỆU ĐẶC TẢ YÊU CẦU NGƯỜI DÙNG “HỆ THỐNG QUẢN LÝ VÀ ĐẶT LỊCH SÂN PICKLEBALL”

1. Giới thiệu

Tài liệu này trình bày các Yêu cầu người dùng (User Requirements) cho dự án phát triển hệ thống Đặt sân PickleBook. Hệ thống được xây dựng nhằm cung cấp một nền tảng trên thiết bị di động, giúp người dùng dễ dàng tìm kiếm, xem vị trí sân qua bản đồ, đặt lịch thuê sân và quản lý các đơn đặt của mình. Mục đích của tài liệu là làm rõ những mong đợi và kỳ vọng của người dùng cuối (khách hàng, chủ sân, admin), là cơ sở để đội ngũ phát triển và các bên liên quan cùng hiểu và thực hiện dự án.

2. Các Loại Yêu Cầu

Hệ thống cần đáp ứng cả Yêu cầu chức năng (các tính năng cụ thể) và Yêu cầu phi chức năng (các yếu tố về chất lượng hệ thống).

2.1. Yêu cầu Chức năng (Functional Requirements - FR)

Các yêu cầu này mô tả những tính năng và hành vi mà hệ thống phải thực hiện để đáp ứng nhu cầu của người dùng.

2.1.1. Yêu cầu liên quan đến Người dùng và Tài khoản

- Đăng ký và Đăng nhập:
 - FR-1.1: Hệ thống phải cho phép người dùng mới đăng ký tài khoản (Khách hàng hoặc Chủ sân) bằng cách cung cấp email, tên đầy đủ và mật khẩu.
 - FR-1.2: Hệ thống phải xác thực thông tin đăng nhập của người dùng đã có tài khoản.

- FR-1.3: Hệ thống phải có chức năng "Quên mật khẩu", cho phép người dùng đặt lại mật khẩu qua email.
- Quản lý Tài khoản cá nhân:
 - FR-1.4: Người dùng phải có khả năng xem và chỉnh sửa thông tin cá nhân của họ, bao gồm đổi mật khẩu và cập nhật ảnh đại diện.
 - FR-1.5: Người dùng phải có thể xem lịch sử đặt sân của họ, bao gồm các đơn đã đặt, trạng thái đơn (pending/confirmed/cancelled).

2.1.2. Yêu cầu liên quan đến Sân và Tìm kiếm

- Tìm kiếm và Lọc sân:
 - FR-2.1: Hệ thống phải cho phép người dùng tìm kiếm sân theo tên hoặc địa điểm.
 - FR-2.2: Hệ thống phải hiển thị bản đồ (Google Maps) có ghim vị trí của tất cả các sân xung quanh.
- Hiển thị Thông tin sân:
 - FR-2.3: Hệ thống phải hiển thị một trang chi tiết cho mỗi sân, bao gồm ảnh, giá thuê, địa chỉ và các tiện ích đi kèm.
 - FR-2.4: Chủ sân phải có khả năng thêm sân mới (kéo thả marker trên bản đồ để lưu tọa độ), chỉnh sửa thông tin sân và bật/tắt trạng thái hoạt động của sân.

2.1.3. Yêu cầu liên quan đến Đặt lịch và Xử lý đơn đặt

- Quản lý Đặt sân (Khách hàng):
 - FR-3.1: Khách hàng phải có khả năng xem lịch trống của sân theo ngày (hiển thị slot giờ).
 - FR-3.2: Khách hàng có thể đặt sân bằng cách chọn ngày, giờ bắt đầu và giờ kết thúc.
 - FR-3.3: Khách hàng có thể hủy đặt sân khi đơn đặt còn ở trạng thái chờ xác nhận (pending).
- Quản lý Đơn đặt (Chủ sân):

- FR-3.4: Chủ sân phải có khả năng xem tất cả đơn đặt của sân mình.
- FR-3.5: Chủ sân có quyền xác nhận hoặc từ chối đơn đặt của khách hàng.
- FR-3.6: Hệ thống phải có thông báo trong app khi đặt sân thành công hoặc bị từ chối.

2.1.4. Yêu cầu liên quan đến Quản trị hệ thống (Admin)

- FR-4.1: Quản trị viên phải có khả năng xem danh sách tất cả người dùng và khoá/mở khoá tài khoản.
- FR-4.2: Quản trị viên phải có khả năng xem danh sách sân và duyệt/từ chối đăng ký sân mới của Chủ sân.

2.2. Yêu cầu Phi chức năng (Non-Functional Requirements - NFR)

Các yêu cầu này mô tả các thuộc tính chất lượng của hệ thống, không phải là tính năng cụ thể.

- Hiệu suất (performance):
 - NFR-1.1: Tốc độ tải trang: Màn hình danh sách sân và bản đồ Google Maps phải tải hoàn toàn trong vòng dưới 3 giây.
 - NFR-1.2: Thời gian phản hồi: Hệ thống phải gửi thông báo cập nhật trạng thái đơn đặt (real-time) cho chủ sân trong vòng chưa đầy 2 giây.
- Khả năng sử dụng (usability):
 - NFR-2.1: Giao diện trực quan: Có màn hình Splash (logo animation) và Onboarding (giới thiệu app) khi mở lần đầu.
 - NFR-2.2: Điều hướng thông minh: Hệ thống cung cấp màn hình Home/Dashboard tương ứng với 3 role riêng biệt (Customer, Owner, Admin) ngay sau khi đăng nhập.
- Bảo mật (security):
 - NFR-3.1: Bảo vệ dữ liệu người dùng: Mật khẩu và thông tin đăng nhập phải được bảo mật và mã hóa tự động qua Firebase Auth.
- Độ tin cậy (reliability):

- NFR-4.1: Thời gian hoạt động (Uptime): Cơ sở dữ liệu Firebase Firestore đảm bảo hoạt động liên tục 99.9% mỗi tháng.
- Khả năng mở rộng (scalability):
 - NFR-5.1: Mở rộng cơ sở dữ liệu: Cấu trúc cơ sở dữ liệu phải hỗ trợ thêm mới hàng ngàn sân và tài khoản mà không làm giảm tốc độ truy vấn.

3. Yêu cầu Nghiệp vụ (Domain Requirements)

Đây là các quy tắc kinh doanh và ràng buộc mà hệ thống phải tuân theo.

- Quản lý khung giờ trống:
 - Khi khách đặt, hệ thống bắt buộc phải so sánh giờ bắt đầu (startTime) và giờ kết thúc (endTime) để tránh đặt trùng khung giờ đã có người đặt trước.
- Quản lý trạng thái đơn đặt:
 - Trạng thái đơn đặt phải di chuyển theo luồng quy định: từ chờ xác nhận (pending) sang đã xác nhận (confirmed), bị từ chối (rejected), hoặc đã hủy (cancelled).
- Phân quyền dữ liệu:
 - Chủ sân chỉ được phép xem và quản lý các đơn đặt thuộc về sân do chính họ tạo ra (dựa trên ownerId).

4. Phi yêu cầu (Non-Requirements)

Phần này xác định những gì nằm ngoài phạm vi của dự án.

- [1] Không tích hợp online mobile banking: Hệ thống trong Giai đoạn 1 không xử lý các giao dịch quét mã QR hay chuyển khoản tự động qua ngân hàng.
- [2] Không bán sản phẩm vật lý: Hệ thống chỉ cung cấp dịch vụ đặt giờ thuê sân, không kinh doanh dụng cụ thể thao (vợt, bóng, trang phục).
- [3] Không hỗ trợ nhiều ngôn ngữ: Hệ thống chỉ được phát triển với ngôn ngữ tiếng Việt.

9.2.2. Bài tập 3b – Đặc tả yêu cầu phần mềm

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

DỰ ÁN: HỆ THỐNG ĐẶT SÂN PICKLEBOOK

1. Giới thiệu

Tài liệu này cụ thể hóa các yêu cầu kỹ thuật cho hệ thống Đặt sân PickleBook, phục vụ cho việc thiết kế, lập trình và kiểm thử. Hệ thống tập trung vào tính năng định vị bản đồ và quản lý luồng đặt sân trực tuyến.

2. Mô tả tổng quát

Hệ thống là một ứng dụng di động (Android Native) cho phép khách hàng thực hiện quy trình tìm sân, xem lịch trống và đặt sân.

- Tác nhân (Actors): Khách hàng (Customer), Chủ sân (Owner), Quản trị viên (Admin).

3. Yêu cầu chức năng chi tiết

3.1. Quản lý Tài khoản & Xác thực

- SRS-F01 (Đăng ký/Đăng nhập): Ứng dụng cung cấp form nhập liệu. Sử dụng Firebase Auth để xử lý Đăng nhập bằng Email/Password. Cần validation form đầy đủ (email hợp lệ, mật khẩu ≥ 6 ký tự).
- SRS-F02 (Phân quyền): Lưu role trong collection users/{uid} trên Firestore. Sau khi đăng nhập, hệ thống đọc role để điều hướng đến HomeActivity tương ứng.
- SRS-F03 (Lịch sử đặt sân): Truy vấn collection bookings lọc theo customerId để hiển thị danh sách đơn đặt của người dùng.

3.2. Quản lý Sân & Bản đồ

- SRS-F04 (Bản đồ): Tích hợp Google Maps Android SDK (play-services-maps, play-services-location). Cấu hình API_KEY trong AndroidManifest.xml.
- SRS-F05 (Hiển thị sân): Truy vấn collection courts. Lấy các trường latitude và longitude để khởi tạo các Marker ghim trên MapView.
- SRS-F06 (Chỉ đường): Sử dụng Intent Android gọi mở Google Maps app để chỉ đường từ vị trí hiện tại đến tọa độ đích của sân.

3.3. Đặt lịch & Xử lý đơn đặt

- SRS-F07 (Logic đặt sân): Khi khách bấm đặt, tạo Document trong collection bookings với các trường: courtId, customerId, ownerID, date, startTime, endTime, status = 'pending'.
- SRS-F08 (Real-time update): Sử dụng hàm addSnapshotListener() của Firestore trên collection bookings để Chủ sân thấy đơn mới ngay lập tức mà không cần F5 (tải lại).
- SRS-F09 (Thông báo): Khi trạng thái đơn thay đổi, ghi dữ liệu vào collection notifications/{notifId}, kích hoạt hiển thị thông báo trong app cho Khách hàng/Chủ sân.

3.4. Tương tác Quản trị

- SRS-F10 (Quản lý User & Sân): Tại Admin Panel, truy vấn danh sách users và courts từ Firestore. Cho phép Admin cập nhật trường status của User thành bị khóa, hoặc duyệt đăng ký sân.

4. Yêu cầu phi chức năng (Non-Functional Requirements)

Mã số	Đặc tính	Chi tiết kỹ thuật
SRS-N01	Hiệu suất	Phản hồi truy vấn Firestore < 1s. Tải bản đồ Google Maps < 3s.
SRS-N02	Ràng buộc nền tảng	Tương thích và hoạt động ổn định trên thiết bị Android (Native).
SRS-N03	Bảo mật	Firebase Security Rules chỉ cho phép Chủ sân đọc đơn đặt của sân mình.
SRS-N04	Lưu trữ tệp	Sử dụng Firebase Storage để upload và lưu trữ URL ảnh đại diện, ảnh sân.
SRS-N05	Trải nghiệm UI	Sử dụng BottomNavigationView hoặc DrawerLayout để điều hướng.

5. Quy tắc nghiệp vụ

- BR-01 (Thuật toán kiểm tra lịch): Truy vấn Firestore lấy tất cả booking của courtId + date đó, so sánh khoảng thời gian startTime đến endTime để ngăn chặn khách đặt trùng slot.
- BR-02 (Cập nhật trạng thái): Chủ sân chỉ được phép duyệt đơn đang ở trạng thái 'pending'. Khi chuyển sang 'confirmed' hoặc 'rejected', khách hàng không thể tự hủy đơn nữa.

6. Ràng buộc & Phạm vi

- Không có cổng thanh toán online mobile banking trực tiếp trong app.
- Ngôn ngữ giao diện mặc định: Tiếng Việt.

BÀI 10. NGUYÊN TẮC LÀM VIỆC CỦA GIT

10.1. Yêu cầu bài tập

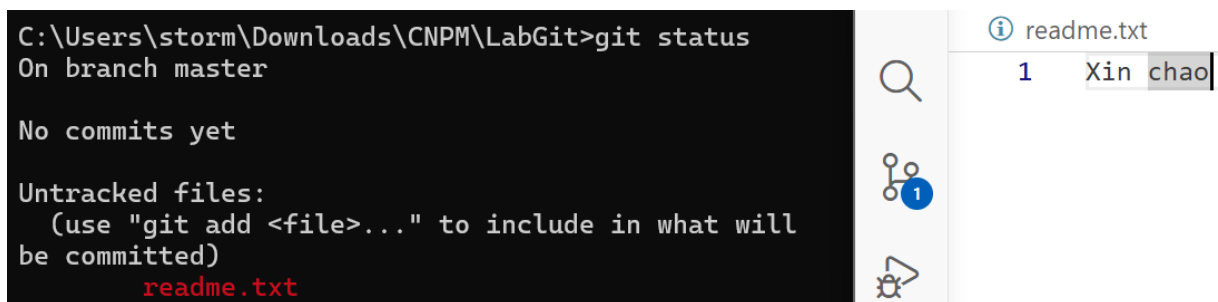
- Bài tập 6a: Thực hành quy trình luân chuyển dữ liệu qua 3 khu vực của Git.
- Bài tập 6b: Phân tích trạng thái tệp tin và lệnh so sánh (git diff).
- Bài tập 6c: Thiết kế quy trình xử lý sự cố bảo mật và khôi phục dữ liệu.
- Bài tập 6d: Tư duy suy luận về cơ chế lưu trữ của Git.

10.2. Báo cáo chi tiết

10.2.1. Bài tập 6a - Luân chuyển dữ liệu qua 3 khu vực

Quy trình quản lý phiên bản trong Git diễn ra tuần tự qua ba khu vực: Thư mục làm việc (Working Directory) -> Khu tạm (Staging Area) -> Kho chứa (Repository). Quá trình làm việc thực tế trên dự án LabGit với tệp readme.txt trải qua các bước sau:

- Bước 1 & 2: Khởi tạo kho chứa cục bộ cho thư mục LabGit và tạo một tệp tin văn bản mới. Lúc này, Git nhận diện tệp nằm ngoài cơ sở dữ liệu theo dõi.



Hình 10.1. Khởi tạo kho chứa Git và tạo tệp tin readme.txt mới (trạng thái Untracked)

- Bước 3 & 4: Đưa tệp tin vào Khu tạm (tệp index) để chuẩn bị lưu trữ, sau đó thử gỡ bỏ nó về lại trạng thái ban đầu để thấy rõ tính linh hoạt của vùng đệm này.

```
C:\Users\storm\Downloads\CNPM\LabGit>git add readme.txt

C:\Users\storm\Downloads\CNPM\LabGit>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   readme.txt
```

Hình 10.2. Đưa tệp tin readme.txt vào Khu tạm bằng lệnh git add (trạng thái Staged)

```
C:\Users\storm\Downloads\CNPM\LabGit>git rm --cached readme.txt
rm 'readme.txt'

C:\Users\storm\Downloads\CNPM\LabGit>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        readme.txt
```

Hình 10.3. Gỡ bỏ tệp tin readme.txt khỏi Khu tạm bằng lệnh git rm --cached

- Bước 5: Sau khi gỡ bỏ, đưa tệp trở lại Khu tạm và thực thi lệnh lưu trữ vĩnh viễn vào Kho chứa .git.

```
C:\Users\storm\Downloads\CNPM\LabGit>git add readme.txt

C:\Users\storm\Downloads\CNPM\LabGit>git commit -m "Khoi tao du an voi tap tin readme"
[master (root-commit) 6a21d6e] Khoi tao du an voi tap tin readme
1 file changed, 1 insertion(+)
create mode 100644 readme.txt

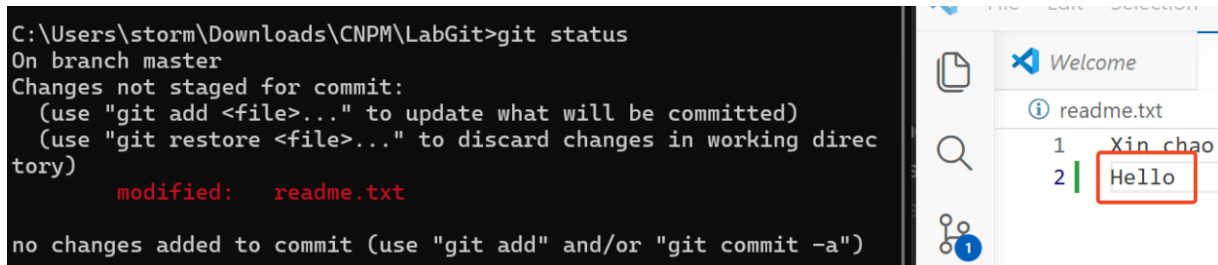
C:\Users\storm\Downloads\CNPM\LabGit>git status
On branch master
nothing to commit, working tree clean
```

Hình 10.4. Lưu tệp tin readme.txt vào Kho chứa bằng lệnh git commit.

10.2.2. Bài tập 6b - Trạng thái tệp tin và lệnh git diff

- Tình huống 1: Mở tệp readme.txt, thêm một dòng nội dung mới và lưu lại. Khi chạy git status, tệp xuất hiện ở mục *Changes not staged for commit* (màu đỏ).

Phân tích: Sự khác nhau nằm ở "dấu vết" trong cơ sở dữ liệu. Tập mới tạo chưa từng tồn tại trong Git nên bị xếp vào *Untracked*. Ngược lại, tệp readme.txt đã từng được *commit*, nghĩa là Git đã giữ một bản ghi của nó. Khi nội dung hiện tại sai lệch với bản ghi cũ trong Kho chứa (thông qua mã SHA-1), Git xếp nó vào nhóm *Modified* (đã sửa đổi nhưng chưa đưa vào Khu tạm).



The image shows a terminal window on the left and a file explorer on the right. The terminal window displays the output of the `git status` command, indicating that the file `readme.txt` is modified. The file explorer on the right shows the contents of `readme.txt`, which are "Xin chào" on line 1 and "Hello" on line 2. The word "Hello" is highlighted with a red box.

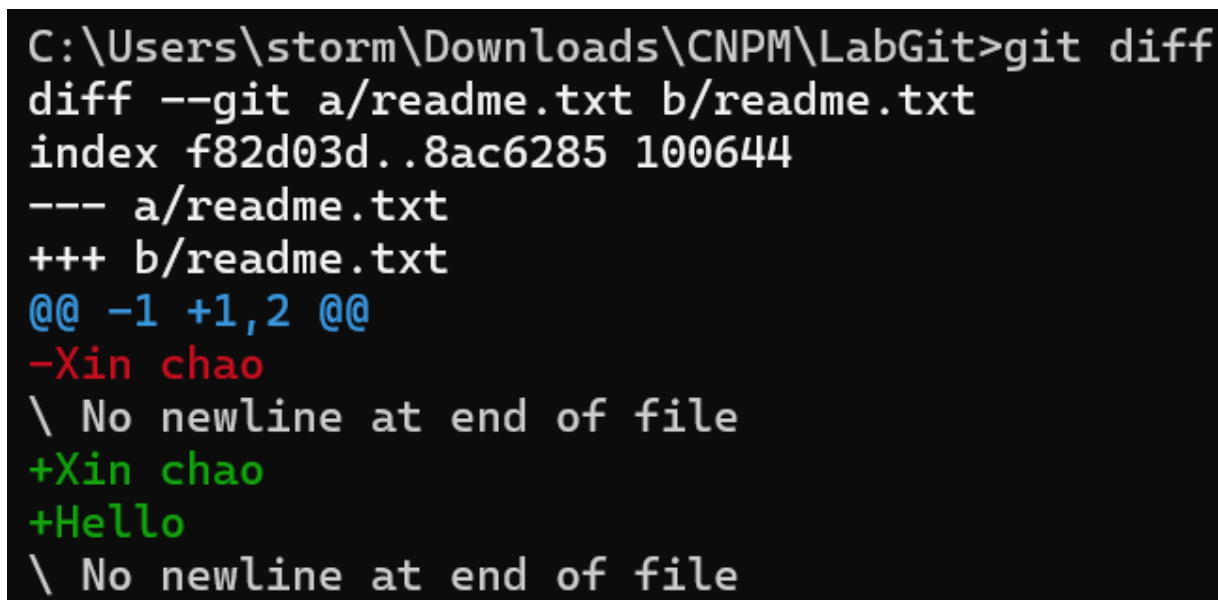
```
C:\Users\storm\Downloads\CNPM\LabGit>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   readme.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Hình 10.5. Thay đổi nội dung tệp tin readme.txt để chuyển sang trạng thái Modified

- Tình huống 2: Chạy lệnh `git diff` để xem chi tiết sự thay đổi.

Phân tích: Lệnh `git diff` mặc định so sánh sự khác biệt giữa Thư mục làm việc (Working Directory) và Khu tạm (Staging Area). Lệnh này hiển thị chính xác dòng "Hello" vừa được thêm vào tại Thư mục làm việc nhưng chưa được đánh dấu (`git add`).



The image shows a terminal window displaying the output of the `git diff` command. The output shows the difference between the current working directory and the staging area. It indicates that the file `readme.txt` has been modified, and the changes are summarized as follows: a newline at the end of the file, followed by the text "Xin chào" and "Hello".

```
C:\Users\storm\Downloads\CNPM\LabGit>git diff
diff --git a/readme.txt b/readme.txt
index f82d03d..8ac6285 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,2 @@
-Xin chào
\ No newline at end of file
+Xin chào
+Hello
\ No newline at end of file
```

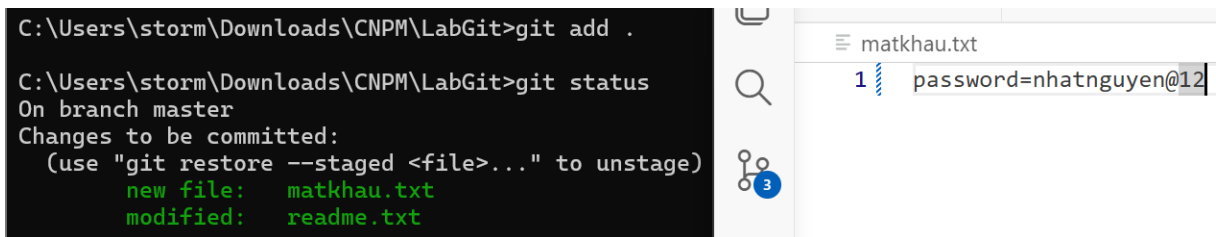
Hình 10.6. So sánh sự thay đổi nội dung giữa Thư mục làm việc và Khu tạm

10.2.3. Bài tập 6c - Xử lý sự cố và khôi phục dữ liệu

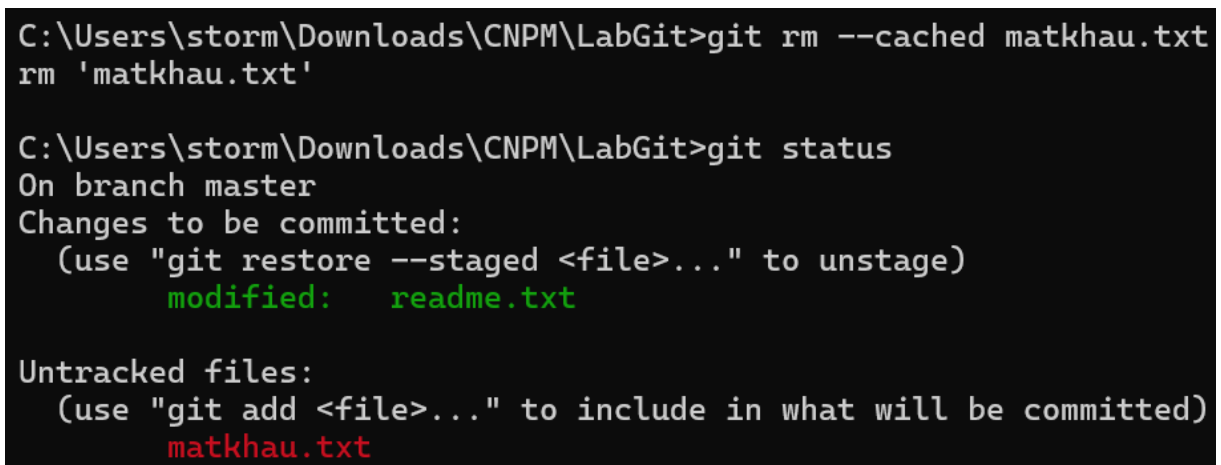
Tình huống 1: Xử lý sự cố "Lỡ tay đưa nhầm tập tin quan trọng"

- Giả lập sự cố: Tạo tệp `matkhau.txt` và lỡ tay gõ `git add` . đưa toàn bộ vào Khu tạm.

- Quy trình xử lý: Sử dụng lệnh `git rm --cached matkhau.txt` để rút tệp ra khỏi Khu tạm.
- Giải thích: Xóa vật lý tệp `matkhau.txt` sẽ làm mất vĩnh viễn dữ liệu cá nhân. Thay vào đó, lệnh `git rm --cached` can thiệp thẳng vào tệp index, gỡ bỏ liên kết theo dõi của tệp này ra khỏi Khu tạm, trả nó về trạng thái an toàn (Untracked) tại Thư mục làm việc.



Hình 10.7. Đưa toàn bộ tệp tin, bao gồm cả tệp nhạy cảm (`matkhau.txt`), vào Khu tạm



Hình 10.8. Gỡ bỏ tệp tin nhạy cảm (`matkhau.txt`) khỏi Khu tạm để bảo toàn dữ liệu gốc

Tình huống 2: Xử lý sự cố "Khôi phục trạng thái cũ"

- Quy trình xử lý: Sử dụng lệnh `git restore readme.txt` để xóa bỏ những sửa đổi sai lầm ở Tình huống 1.
- Giải thích: Lệnh này trích xuất bản ghi (snapshot) gần nhất của tệp `readme.txt` đang nằm an toàn trong Kho chứa và "ghi đè" ngược trở lại Thư mục làm việc. Kho chứa đóng vai trò là "nguồn dữ liệu tin cậy" vì nó lưu trữ cấu trúc nguyên bản, đã chốt tính toàn vẹn bằng mã SHA-1 tại thời điểm commit.

```
C:\Users\storm\Downloads\CNPM\LabGit>git restore readme.txt

C:\Users\storm\Downloads\CNPM\LabGit>git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
      matkhau.txt
```

Hình 10.8. Khôi phục nội dung tệp tin *readme.txt* về bản lưu gần nhất trong Kho chứa.

10.2.4. Bài tập 6d - Cơ chế lưu trữ của Git

- Nếu người A sửa tệp tin nhưng chưa *git add*, người B có thấy được thay đổi đó trong Kho chứa không?

Người B không thể thấy. Các chỉnh sửa của người A lúc này chỉ mang tính cục bộ, lưu trên ổ cứng vật lý (Thư mục làm việc) của máy tính người A. Chúng chưa được đánh dấu (Staged) và chưa được lưu vào cơ sở dữ liệu chung (Committed) để đẩy lên server.

- Tại sao Git lại cần "Khu tạm" (Staging Area) mà không cho phép Commit thẳng từ Thư mục làm việc?

Khu tạm (Staging Area) hoạt động như một "vùng đệm phân loại". Nó giải quyết bài toán: Một lập trình viên sửa cùng lúc 10 tệp tin cho 2 tính năng hoàn toàn riêng biệt. Nhờ Khu tạm, họ có quyền nhặt 5 tệp của tính năng A đưa vào trước để tạo *Commit số 1*; sau đó mới đưa 5 tệp còn lại vào tạo *Commit số 2*. Nếu commit thẳng từ Thư mục làm việc, mã nguồn sẽ bị gom chung thành một mớ hỗn độn, vi phạm nguyên tắc "Mỗi commit giải quyết một công việc duy nhất".

BÀI 11. THIẾT KẾ KIẾN TRÚC PHẦN MỀM

11.1. Yêu cầu bài tập

- Bài 5a. Dựa trên URS ứng dụng của bạn (nhóm), hãy phân tích và thiết kế theo kiến trúc 3-tầng. Bạn có thể tự phân tích, thiết kế hoặc nhờ sự hỗ trợ của chatbot.
- Bài 5b. Dựa trên URS ứng dụng của bạn (nhóm), hãy phân tích và thiết kế theo kiến trúc vi dịch vụ. Bạn có thể tự phân tích, thiết kế hoặc nhờ sự hỗ trợ của chatbot.

11.2. Báo cáo chi tiết

11.2.1. Bài tập 5a - Kiến trúc 3-tầng cho dự án *PickleBook*

Dựa trên tài liệu URS và kế hoạch phát triển thực tế, hệ thống PickleBook được phân rã thành kiến trúc 3 tầng độc lập. Việc phân lớp này giúp tách biệt rạch ròi giữa giao diện hiển thị, logic xử lý và cơ sở dữ liệu (tận dụng hệ sinh thái Firebase).

- Tầng Trình bày (Presentation Tier - Tầng Giao diện):
 - Nhiệm vụ: Tương tác trực tiếp với người dùng (Customer, Owner, Admin). Nhận thao tác chạm, hiển thị danh sách sân, render bản đồ Google Maps và form đặt lịch.
 - Công nghệ: Mã nguồn Native Android (Java/XML). Sử dụng các thành phần UI như RecyclerView (danh sách), BottomNavigationView (điều hướng), MapView (bản đồ) và BottomSheet (thông tin chi tiết).
- Tầng Logic nghiệp vụ (Business Logic Tier - Tầng Ứng dụng):
 - Nhiệm vụ: Đóng vai trò "bộ não" điều phối dữ liệu. Xử lý logic phân quyền người dùng (Role-based routing), xác thực tính hợp lệ của form (validate email/mật khẩu), và đặc biệt là thực thi thuật toán đối chiếu thời gian (Booking Conflict Resolution) để chặn đặt trùng lịch trước khi đẩy dữ liệu đi.
 - Công nghệ: Các lớp logic trong Android (ViewModels, Controllers, Repositories) kết hợp với Firebase Security Rules (chặn quyền truy cập sai luồng) và Firebase Authentication (đảm nhận logic mã hóa mật khẩu và cấp token).
- Tầng Dữ liệu (Data Tier):

- Nhiệm vụ: Lưu trữ toàn bộ trạng thái của hệ thống một cách an toàn, đồng bộ hóa dữ liệu thời gian thực (real-time) tới các thiết bị khách.
- Công nghệ: Sử dụng hệ thống backend-as-a-service của Google. Cụ thể: Firebase Cloud Firestore lưu trữ 4 collections cốt lõi (users, courts, bookings, notifications), và Firebase Storage lưu trữ các tệp vật lý (ảnh đại diện, ảnh sân).

11.2.2. Bài tập 5b: Kiến trúc Vi dịch vụ cho dự án PickleBook

Trong tương lai, khi ứng dụng PickleBook mở rộng quy mô toàn quốc với hàng triệu truy cập đồng thời, hệ thống nguyên khối (hoặc phụ thuộc hoàn toàn vào một BaaS như Firebase) có thể gặp nút thắt cổ chai. Dưới đây là bản thiết kế chuyển đổi hệ thống sang kiến trúc Microservices để tối ưu khả năng chịu tải:

Thay vì dồn tất cả vào một khối, hệ thống được xé nhỏ thành 4 dịch vụ (services) độc lập, chạy trên các tiến trình và cơ sở dữ liệu riêng biệt:

- User & Auth Service (Dịch vụ Tài khoản):
 - Nhiệm vụ: Chỉ chịu trách nhiệm quản lý đăng ký, đăng nhập, cấp phát Token (JWT) và lưu trữ hồ sơ cá nhân.
 - Công nghệ đề xuất: Node.js (Express) kết hợp với cơ sở dữ liệu MongoDB (tốc độ đọc ghi JSON nhanh).
- Court & Maps Service (Dịch vụ Quản lý sân & Bản đồ):
 - Nhiệm vụ: Xử lý logic tìm kiếm sân, lọc dữ liệu theo khoảng cách địa lý và tọa độ GPS, hiển thị thông tin sân bãi.
 - Công nghệ đề xuất: Python (Django) kết hợp cơ sở dữ liệu PostgreSQL (hỗ trợ thư viện PostGIS chuyên xử lý tọa độ bản đồ cực mạnh).
- Booking Service (Dịch vụ Đặt lịch):
 - Nhiệm vụ: Dịch vụ lõi, xử lý thuật toán xếp lịch, giữ chỗ (lock slot) trong môi trường truy cập đồng thời cao để chống trùng lặp đơn đặt.
 - Công nghệ đề xuất: Golang (chạy đa luồng cực tốt) kết hợp Redis (cache bộ nhớ đệm để kiểm tra giờ trống nhanh tính bằng mili-giây) và MySQL (đảm bảo tính toàn vẹn giao dịch ACID).

- Notification Service (Dịch vụ Thông báo):
 - Nhiệm vụ: Lắng nghe các sự kiện (ví dụ: Đặt sân thành công, Chủ sân hủy đơn) và đẩy thông báo (Push Notification) đến điện thoại.
 - Công nghệ đề xuất: Node.js kết hợp Message Queue (RabbitMQ) để xếp hàng hàng vạn thông báo gửi đi cùng lúc thông qua FCM (Firebase Cloud Messaging).
- Cơ chế giao tiếp: Các dịch vụ trên hoàn toàn độc lập. Ứng dụng Android sẽ không gọi trực tiếp vào từng database nữa mà gọi thông qua một cổng duy nhất gọi là API Gateway. Gateway sẽ tự động phân luồng: user bấm tìm sân -> gọi Court Service; user bấm thanh toán -> gọi Booking Service. Nếu Dịch vụ Thông báo bị sập mạng, khách hàng vẫn tìm và đặt sân được bình thường (Khả năng chịu lỗi - Fault Isolation).

BÀI 12. LÀM QUEN VỚI DOCKER

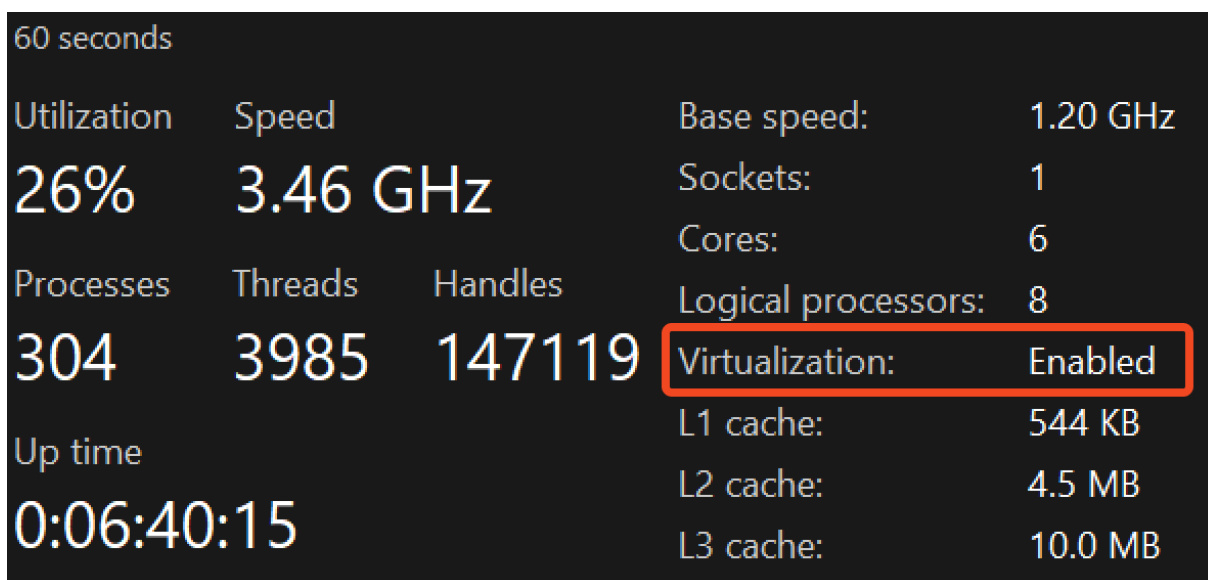
12.1. Yêu cầu bài tập

Bài tập 1a: Tải, cài đặt Docker Desktop và cấu hình môi trường WSL 2 trên máy tính cá nhân.

12.2. Báo cáo chi tiết

Việc cài đặt Docker trên môi trường Windows đòi hỏi hệ thống phải đáp ứng các tiêu chuẩn về ảo hóa phần cứng. Quá trình thiết lập được thực hiện qua các bước sau:

- Bước 1: Kiểm tra nền tảng ảo hóa phần cứng (Virtualization)
Mở Task Manager (tổ hợp phím Ctrl + Shift + Esc), chuyển sang tab *Performance* và chọn mục *CPU* để xác nhận hệ thống đã kích hoạt sẵn tính năng ảo hóa từ BIOS.



60 seconds				
Utilization	Speed		Base speed:	1.20 GHz
26%	3.46 GHz		Sockets:	1
			Cores:	6
Processes	Threads	Handles	Logical processors:	8
304	3985	147119	Virtualization:	Enabled
Up time			L1 cache:	544 KB
0:06:40:15			L2 cache:	4.5 MB
			L3 cache:	10.0 MB

Hình 12.1. Kiểm tra tính năng ảo hóa đã được kích hoạt trên hệ thống Windows.

- Bước 2: Cài đặt phần mềm và cấu hình WSL 2 Tải bộ cài đặt trực tiếp từ trang chủ Docker và chạy tệp thực thi. Trong màn hình thiết lập, tích chọn mục *"Use WSL 2 instead of Hyper-V"* để sử dụng kiến trúc Windows Subsystem for Linux thế hệ mới, giúp Docker hoạt động với hiệu năng tối ưu nhất.

Bước 3: Khởi chạy và xác nhận cài đặt (Kiểm tra Docker Engine) Khởi động ứng dụng Docker Desktop và chờ hệ thống tải dịch vụ (biểu tượng trên system tray hiển thị trạng thái đang chạy). Tiếp theo, mở cửa sổ dòng lệnh (CMD hoặc PowerShell) và gõ lệnh kiểm tra phiên bản để xác nhận sự tồn tại của cả hai thành phần lõi: Docker Client (người điều khiển) và Docker Server/Daemon (người thực hiện).

```
C:\Users\storm>docker version
Client:
Version:           29.4.1
API version:       1.54
Go version:        go1.26.2
Git commit:        055a478
Built:             Mon Apr 20 16:35:45 2026
OS/Arch:           windows/amd64
Context:           desktop-linux

Server: Docker Desktop 4.71.0 (225177)
Engine:
Version:           29.4.1
API version:       1.54 (minimum version 1.40)
```

Hình 12.2. Sử dụng lệnh docker version để xác nhận cả Docker Client và Server (Daemon) đều đang hoạt động thành công.

BÀI 13. THIẾT KẾ CHI TIẾT

13.1. Yêu cầu bài tập

Bài tập: Lựa chọn một chức năng cốt lõi trong ứng dụng của nhóm và thực hiện Thiết kế chi tiết (Detailed Design) dựa trên 4 nội dung: Dữ liệu, Logic xử lý, Giao diện lập trình (API) và Phân cấp lớp (OOP).

13.2. Báo cáo chi tiết

Dựa trên kiến trúc hệ thống của ứng dụng PickleBook, nhóm tiến hành Thiết kế chi tiết cho chức năng cốt lõi nhất của dự án: Chức năng Đặt sân (Create Booking).

13.2.1. Thiết kế dữ liệu chi tiết

Do dự án sử dụng hệ quản trị cơ sở dữ liệu NoSQL (Firebase Firestore), dữ liệu không lưu theo dạng bảng (Table) mà lưu theo dạng Tài liệu (Document) nằm trong Bộ sưu tập (Collection).

- Collection: bookings
- Cấu trúc Document (JSON format):
 - bookingId: String (Auto-generated UUID của Firestore)
 - courtId: String (Mã ID của sân, tham chiếu đến collection courts)
 - customerId: String (Mã ID của người đặt, tham chiếu đến collection users)
 - ownerId: String (Mã ID của chủ sân)
 - date: String (Định dạng "YYYY-MM-DD")
 - startTime: Number (Lưu dưới dạng giờ thập phân hoặc timestamp)
 - endTime: Number (Lưu dưới dạng giờ thập phân hoặc timestamp)
 - status: String (Mặc định khi khởi tạo là "pending")
 - totalPrice: Number (Tổng tiền tính toán được)

13.2.2. Thiết kế logic xử lý

Sử dụng mã giả (pseudo-code) để mô tả thuật toán kiểm tra trùng lịch (Booking Conflict) cực kỳ quan trọng khi khách hàng bấm nút "Xác nhận đặt sân".

BẮT ĐẦU

NHẬN tham số: courtId, date, newStartTime, newEndTime từ giao diện

Bước 1: Tính toán tiền

LẤY basePrice từ collection courts dựa vào courtId

TÍNH $\text{totalPrice} = \text{basePrice} * (\text{newEndTime} - \text{newStartTime})$

Bước 2: Kiểm tra trùng lịch trên Firestore

TRUY VẤN danh sách các booking CÓ (courtId == tham số) VÀ (date == tham số)

LẶP qua từng booking trong danh sách:

LẤY existingStartTime, existingEndTime của booking đó

KIỂM TRA ĐIỀU KIỆN TRÙNG LẶP:

NẾU ($\text{newStartTime} < \text{existingEndTime}$) VÀ ($\text{newEndTime} > \text{existingStartTime}$):

TRẢ VỀ LỖI "Khung giờ này đã có người đặt, vui lòng chọn giờ khác!"

KẾT THÚC

Bước 3: Ghi dữ liệu

TẠO đối tượng booking mới với status = "pending" và totalPrice vừa tính

GHI dữ liệu vào collection bookings trên Firestore

Bước 4: Chuyển hướng

TRẢ VỀ "Đặt sân thành công, chờ chủ sân xác nhận"

CHUYỂN HƯỚNG sang Màn hình Lịch sử đặt sân (MyBookingsActivity)

KẾT THÚC

13.2.3. Thiết kế giao diện lập trình API

Định nghĩa phương thức (Function) trung gian kết nối giữa UI (Android) và cơ sở dữ liệu (Firestore).

- Tên phương thức: createBookingRequest
- Tham số đầu vào (Input): BookingRequestModel (chứa courtId, customerId, date, startTime, endTime).

- Kiểu dữ liệu trả về (Output - Callback/Coroutine):
 - Thành công: Trả về đối tượng `Result.Success(bookingId)`
 - Thất bại: Trả về đối tượng `Result.Error(Exception)` kèm thông báo lỗi tương ứng.

13.2.4. Thiết kế phân cấp lớp OOP

Do dự án phát triển trên nền tảng Android (Native), kiến trúc lớp được thiết kế theo mô hình MVVM (Model-View-ViewModel) để đảm bảo nguyên lý SOLID.

[1] Lớp Model (Data Class): Booking

- Thuộc tính: Các trường dữ liệu tương tự như thiết kế database (`courtId`, `date`, `startTime`...).
- Bao đóng: Sử dụng data class trong Kotlin để tự động hóa các hàm getter/setter, đảm bảo toàn vẹn dữ liệu.

[2] Lớp View: CreateBookingActivity

- Nhiệm vụ: Nhận thao tác chạm chọn giờ của người dùng, hiển thị thông báo lỗi hoặc chuyển màn hình. Lớp này không chứa logic tính toán tiền hay truy vấn database.
- Mối quan hệ: Quan sát (Observe) sự thay đổi dữ liệu từ lớp ViewModel.

[3] Lớp Logic: BookingViewModel

- Nhiệm vụ: Chứa thuật toán kiểm tra tính hợp lệ của giờ đặt và gọi lớp Repository. Quản lý trạng thái UI (Đang tải, Thành công, Thất bại) thông qua LiveData hoặc StateFlow.

[4] Lớp Truy cập dữ liệu: BookingRepository

- Phương thức: `checkSlotAvailability()` và `insertNewBooking()`.
- Mục đích: Tách biệt hoàn toàn việc viết query Firebase Firestore ra khỏi ViewModel. Nếu sau này đổi cơ sở dữ liệu từ Firebase sang một API Backend riêng, chỉ cần sửa code ở Repository, các lớp khác không bị ảnh hưởng.

BÀI 14. DOCKER IMAGE VÀ DOCKER CONTAINER

14.1. Yêu cầu bài tập

Bài tập: Thực hành toàn bộ quy trình **Build - Ship - Run** trong Docker. Đóng gói một ứng dụng Web tĩnh (HTML chạy trên Nginx) thành Docker Image, sau đó thực hiện chuyển giao (Ship) Image này qua hai phương thức: xuất thành tập tin nén cục bộ (.tar) và đẩy lên kho lưu trữ đám mây (Docker Hub).

14.2. Báo cáo chi tiết

1. Giai đoạn Build: Đóng gói ứng dụng thành Docker Image

Khởi tạo cấu trúc thư mục dự án chứa mã nguồn index.html và kịch bản đóng gói Dockerfile. Sau đó, ra lệnh cho Docker Engine biên dịch chúng thành một Image hoàn chỉnh.

```
C:\Users\storm\Downloads\CNPM\teo-docker-project>docker build -t hello-docker-image:v1
[+] Building 8.6s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 316B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 61B
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:5616878291a2eed594aee8db4dade5878c
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:43a775bcc44222d03f235dee7ce8bcbbcbcaf5ed20ccc9872ff237b1
=> => exporting config sha256:75635eb397c45aa902f134065fd3d448c09d6d2663f8f0465682309e47
=> => exporting attestation manifest sha256:bb0c8facfbe63a245370ee7542459cb3cb4957075e80
=> => exporting manifest list sha256:e8a5ff9be044ee3b54d66404c85e3ab5a934f32d163b55b2a81
=> => naming to docker.io/library/hello-docker-image:v1
=> => unpacking to docker.io/library/hello-docker-image:v1
```

Hình 14.1. Biên dịch thành công Docker Image cục bộ.

2. Giai đoạn Ship (Cách 1): Trích xuất và nạp Image qua tập tin nén (.tar)

Thực hiện trích xuất (save) Image vừa tạo ra thành một tập tin vật lý độc lập để có thể chia sẻ qua USB/Google Drive, sau đó nạp ngược (load) lại vào hệ thống.

```
C:\Users\storm\Downloads\CNPM\teo-docker-project>docker save
-o hello-image.tar hello-docker-image:v1

C:\Users\storm\Downloads\CNPM\teo-docker-project>docker load
-i hello-image.tar
Loaded image: hello-docker-image:v1
```

Hình 14.2. Xuất Docker Image thành tệp hello-image.tar và nạp lại vào Docker Engine

3. Giai đoạn Ship (Cách 2): Đẩy Image lên Docker Hub

Để phân phối Image qua môi trường mạng chuyên nghiệp, thực hiện gắn thẻ (tag) định danh và đẩy (push) Image lên kho lưu trữ Docker Hub.

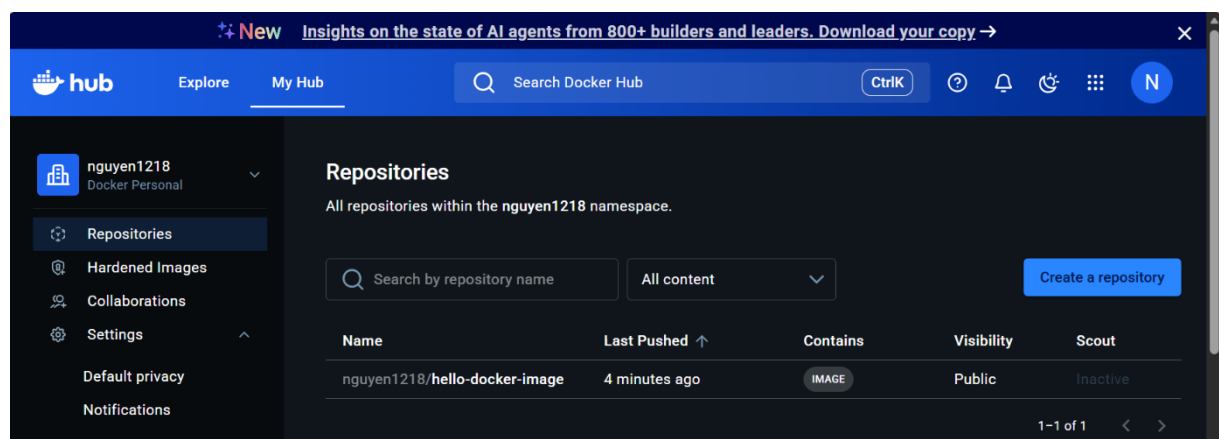
```
hello-docker-image:v1          docker tag hello-docker-image:v1 nguyen1218/he
llo-docker-image:v1

C:\Users\storm\Downloads\CNPM\teo-docker-project>docker login
Authenticating with existing credentials... [Username: nguyen1218]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

C:\Users\storm\Downloads\CNPM\teo-docker-project>docker push nguyen1218/hello-docker-image:v1
The push refers to repository [docker.io/nguyen1218/hello-docker-image]
d96c5464e795: Pushed
aee4e54b3865: Pushed
4a8b0b2a5b19: Pushed
6a0ac1617861: Pushed
82736a35d0e7: Pushed
781ff50d2644: Pushed
0c18b965c652: Pushed
453da7dbc73e: Pushed
612c0c1df4c5: Pushed
583599bb7d38: Pushed
v1: digest: sha256:e8a5ff9be044ee3b54d66404c85e3ab5a934f32d163b55b2a81896ba213ff564 size: 856
```

Hình 14.3. Gắn thẻ định danh tài khoản và đẩy Image lên kho lưu trữ trực tuyến Docker Hub.



Hình 14.4. Khởi chạy Docker Container và kiểm tra thành quả ứng dụng Web hoạt động trên môi trường biệt lập.

KẾT LUẬN

Qua quá trình nghiên cứu, tổng hợp và thực hiện các bài tập trong 14 bài học của môn Công nghệ phần mềm, có thể thấy rằng việc phát triển một hệ thống phần mềm hoàn chỉnh không chỉ dừng lại ở kỹ năng lập trình, mà là sự kết hợp chặt chẽ giữa tư duy hệ thống, quy trình phát triển bài bản và khả năng phối hợp trong môi trường làm việc thực tế. Từ những kiến thức nền tảng như vòng đời phát triển phần mềm, các mô hình phát triển, quản lý phiên bản với Git, cho đến các nội dung nâng cao như phân tích yêu cầu, thiết kế kiến trúc và triển khai hệ thống, bài tiểu luận đã giúp củng cố một cái nhìn toàn diện và thực tiễn về ngành Kỹ thuật phần mềm. Đặc biệt, việc liên hệ trực tiếp với các dự án cá nhân và nhóm như PickleBook, Kermy hay TeoShop đã góp phần làm rõ giá trị ứng dụng của từng khái niệm lý thuyết trong môi trường thực tế. Qua đó, không chỉ nâng cao năng lực chuyên môn, mà còn rèn luyện được tư duy tổ chức công việc, tính kỷ luật và trách nhiệm trong quá trình phát triển sản phẩm. Đây sẽ là nền tảng quan trọng để tiếp tục học tập, nghiên cứu và định hướng phát triển nghề nghiệp trong lĩnh vực Công nghệ phần mềm trong tương lai.